

# Composition Is Where Obfuscation Robustness Breaks

ANONYMOUS AUTHOR(S)

Fine-tuning a code model on obfuscated programs raises its accuracy on those programs by roughly nineteen points. We ask what that number is made of: competence at reading the program, or competence at the surface a particular transform leaves behind. We separate the two with tests that share no machinery — whether the gain survives being stacked with a transform family the model has never seen, and whether it survives the task being run backwards. The field’s existing answers, inference-time routing, weight-space merging, and training on a breadth of transforms, fail both — merging outright, routing by matching breadth wherever the two are measured together. Breadth composes on stacks built from what it saw and loses that advantage the moment an unseen family enters, a gap significant on every one of six models across four lineages; and every method that fits the forward task buys its gain by losing backward competence on the very same programs. Three further instruments locate what is learned instead: the surface a transform leaves, which is exactly what an obfuscator is free to change. That diagnosis names its own remedy. We anchor the model to the one thing a semantics-preserving transform leaves invariant, its own behaviour on the unobfuscated parent of each program, through a paired-consistency objective that trains on exactly the rows breadth trains on. Re-running the same two tests, the anchored model is the only one whose advantage *grows* as input diverges from training, rising above the clean-code control on stacks containing an unseen family, and the only one whose forward gain costs nothing backwards. Its repair holds on six of eight models and reverses on one, and on the unseen family alone it is as robust as clean-code tuning and no more. Adaptation to obfuscated code buys recombination of what the data covered; anchoring recovers what that costs; neither buys invariance.

## 1 Introduction

Code obfuscation is widely used to make software difficult to inspect while preserving its intended functionality. In benign settings, developers employ obfuscation to protect intellectual property, implementation details, and proprietary business logic. In adversarial settings, however, malware authors routinely exploit the same techniques to conceal malicious behavior and frustrate program analysis and reverse engineering. Common transformations include identifier renaming, control-flow restructuring, dead-code insertion, expression rewriting, call indirection, etc. Although such transformations are intended to preserve program semantics, they can substantially obscure how those semantics are expressed in the source code, thereby increasing the difficulty of understanding the behavior of malware code [2, 3, 10, 11].

This difficulty is well established for human analysts. Nguyen *et al.* [7], for example, showed that code obfuscation significantly reduces humans’ accuracy in reasoning about program outputs. Such findings are particularly important for malware analysis, where analysts must often infer the behavior of code whose surface structure has been deliberately engineered to resist comprehension.

Large language models (LLMs) offer a promising alternative for automating such reasoning. Models have demonstrated strong capabilities in code generation, summarization, vulnerability detection, and program understanding [1, 4–6, 8]. However, most evaluations are performed on naturally written programs, where meaningful identifiers, familiar control structures, and common coding idioms provide abundant lexical and structural cues. Obfuscated programs remove or distort many of these cues while retaining the underlying computation, creating a useful stress test of whether an LLM actually reasons about program behavior or instead depends on surface regularities.

Recent studies suggest that this distinction matters. Rong *et al.* [?] showed that semantics-preserving obfuscations can cause substantial degradation in LLM program reasoning, including program-output prediction. Similarly, Nikiema *et al.* [9] reported a statistically significant decline in model performance as the complexity of obfuscation increases. Together, these findings indicate that this limitation is especially problematic for security applications: an AI-assisted malware analyzer should not cease to understand a malicious computation merely due to obfuscation.

An important challenge, however, has received considerably less attention: *real obfuscated programs rarely contain only one transformation*. Obfuscators can combine multiple techniques so that, for example, renamed identifiers are embedded inside flattened control flow, rewritten expressions, and additional irrelevant code. Such *stacked obfuscation* creates a fundamentally harder deployment setting. Success on each transformation independently does not necessarily imply success when those transformations occur together. Consequently, simply demonstrating that a model can be adapted to individual obfuscations is insufficient. For practical malware analysis, the learned capability must also *compose*.

This observation naturally suggests several ways to compose knowledge learned from individual transformations. One could train a specialist for each obfuscation and use a *router* to select the appropriate specialist at inference time. Alternatively, one could combine the specialists in parameter space through *model merging*. A third, and seemingly more direct, strategy is to increase *training breadth* by fine-tuning a single model on examples from many individual obfuscations, hoping that knowledge learned separately will recombine when transformations are stacked. We systematically evaluate these three strategies and find that none provides the desired robustness. The learned router is worth about a point over a random routing policy and nothing over breadth training; merging specialized models performs at or below the clean-code-tuned control; and broader training, although effective on stacks assembled from transformations represented during training, introduces a substantial performance penalty on an unseen obfuscation family. Thus, broader exposure can improve recombination within the training distribution while simultaneously making the model less robust outside that distribution.

Our analysis reveals why. The apparent robustness obtained from obfuscation-specific adaptation does not primarily arise from learning the semantics that remain invariant under transformation. Instead, models learn regularities associated with the *surface artifacts* left by particular obfuscators. Evidence for this behavior appears across multiple analyses: adaptation transfers poorly between distinct mechanisms within the same obfuscation family; the benefit of broad training is substantially stronger when recognizable identifier cues remain available; and improvements disappear when the same programs are queried in a different reasoning direction. Thus, stacking exposes a deeper problem than merely insufficient training coverage: obfuscation-specific fine-tuning can teach the model how particular transformations *look* without teaching it to consistently recover the behavior that those transformations preserve.

This diagnosis suggests a different principle for robust adaptation. Rather than attempting to compose transformation-specific knowledge, we anchor learning to something that is invariant across all such transformations: the model’s behavior on the corresponding *unobfuscated program*. For every obfuscated training program  $x_{\text{obf}}$ , we retain its clean parent  $x_{\text{clean}}$ . A teacher model, tuned for reasoning over clean code, processes  $x_{\text{clean}}$ , while the student processes  $x_{\text{obf}}$ . In addition to the ordinary task loss, we minimize the KL divergence between their output distributions:

$$\mathcal{L} = \text{CE}(y \mid x_{\text{obf}}) + \lambda D_{\text{KL}}(p_{\text{teacher}}(\cdot \mid x_{\text{clean}}) \parallel p_{\text{student}}(\cdot \mid x_{\text{obf}})). \quad (1)$$

The first term exposes the student to obfuscated code, whereas the second explicitly encourages it to preserve the behavioral response of a model reasoning over the semantics-equivalent clean program. Importantly, this objective does not require recognizing which obfuscations are present,

selecting among specialists, merging transformation-specific parameters, or recovering the original source code. Instead, it changes the learning target itself: rather than anchoring adaptation to the surface patterns of individual transformations, it anchors the student to the clean-code behavior that those transformations should preserve.

Our empirical results demonstrate the advantage of this *paired-consistency anchoring*. In contrast to routing, model merging, and breadth-only training, the KL-based objective retains the benefits of learning from diverse obfuscations without paying the corresponding penalty on an unseen obfuscation family. At the 7B scale, the anchored model improves over breadth training by 4.59 percentage points on the unseen family while introducing no degradation on clean code. Additional stress tests show that the resulting model continues to perform well under deeper compositions and transfers to a second programming language. At the same time, our analysis carefully bounds the claim: the method does not create a new ability to solve arbitrary unseen obfuscations beyond that of the clean-code teacher. Rather, it prevents obfuscation adaptation from destroying capabilities the model already possesses. This distinction is important: robust reasoning over obfuscated programs requires not only acquiring adaptation gains, but also preserving semantic competence as the surface form of the program changes.

*Contributions.* This paper makes the following main contributions:

**1. A two-test diagnostic for whether obfuscation adaptation generalises or memorises, and its verdict on the existing composition strategies.** We ask whether competence learned on individual semantics-preserving transformations survives leaving the distribution it was fit to, in two independent ways: being stacked with a transformation family the model has never seen, and being asked backwards (given a return value, produce a call that yields it). Inference-time routing, weight-space merging, and broad multi-transformation training all fail both. Breadth’s advantage on stacks of seen transformations vanishes once an unseen family enters — a gap significant on six of six models across four lineages — and every method that fits the forward task buys its gain by losing backward competence on the same programs.

**2. An empirical characterisation of what adaptation learns instead.** With complementary analyses across transformation mechanisms, identifier dependence, and confidence behaviour, we show that obfuscation adaptation predominantly learns the surface regularities a transformation leaves behind rather than an invariance to semantics-preserving change. Because that surface is exactly what an obfuscator is free to alter, the diagnosis names its own remedy.

**3. A clean-behaviour anchoring objective, derived from that diagnosis.** We introduce a paired-consistency objective that anchors an obfuscated-code student to a clean-code-tuned teacher reading the unobfuscated parent of the same program — the one thing a semantics-preserving transformation leaves invariant. The objective trains on exactly the rows breadth training uses, so breadth is its data-matched control. Across eight models and four lineages it removes breadth’s unseen-family degradation on six and reverses on one, and its clean-code cost is absent on five; we report both qualifiers rather than the unconditional claim.

**4. The same two tests, re-run on the anchored model, with the bound stated.** On a new divergence ladder of stacks that contain an unseen transformation family — a stimulus absent from prior work — the anchored model’s advantage *grows* as the input diverges from training, rising above the clean-code control where breadth falls, and it is the only supervised-fine-tuning arm whose forward gain costs nothing backwards. It does not acquire backward competence, and on the unseen family alone it is as robust as clean-code tuning and no more: anchoring repairs what adaptation breaks rather than creating new invariance.

## 2 Research Questions

Fine-tuning a code model on obfuscated programs raises its accuracy on those programs by roughly nineteen points. Every question below asks what that number is made of. Adaptation can raise forward accuracy by teaching the model to *read* the program better, or by teaching it the surface a particular transform leaves behind; only the first deserves to be called robustness. We separate them with two tests that share no machinery, committed before any method was on the table:

- **Composition.** Does competence on single transforms survive being stacked with a transform family the model has never seen?
- **Reversal.** Does it survive the task being run backwards — given a return value, produce a call that yields it? No adapter is trained on this direction, so any backward gain is transferred program understanding rather than task fitting.

**RQ1: Do standard adaptation methods generalise, or memorise?** We apply both tests to the field’s existing strategies — inference-time routing over per-transform specialists, weight-space merging of those specialists, and broad multi-transform fine-tuning — treated as baselines rather than proposals.

**RQ2: What is memorised?** Given the baselines’ failure, this question locates what adaptation latches onto instead, because that determines the remedy.

**RQ3: Does anchoring to clean-code behaviour repair what adaptation breaks?** The diagnosis names its own remedy: anchor the model to its own behaviour on the unobfuscated parent of each program. We derive a paired-consistency objective, isolate its mechanism by ablating the teacher, and measure it across eight models and four lineages.

**RQ4: Does the anchored model generalise?** We re-run RQ1’s two tests on the anchored model and state the bound: a new capability on unseen obfuscation, or a preservation mechanism that protects the teacher’s competence from surface-level change.

## 3 Setup

### 3.1 Models

Table 1 lists the panel: eight instruct models from four lineages (Meta, BigCode, Google, IBM), code-specialised and general, 7B to 34B, plus one pretrained checkpoint used only for the one-shot baseline. Every adapter is a LoRA with the same rank and target modules on every model, and one prompt builder serves training, evaluation and attention extraction.

Table 1. **The model panel.** Four lineages, code-specialised and general instruct models, 7B to 34B. *Chat template: system* accepts a system role; *merged* folds the system text into the user turn; *plain* has no template and is used for the one-shot baseline only. Parameter counts are nominal. Every adapter is a LoRA with the same rank and target modules on every model.

| Model               | Checkpoint                           | Lineage | Type       | Params (B) | Layers | Hidden | Chat template |
|---------------------|--------------------------------------|---------|------------|------------|--------|--------|---------------|
| CodeLlama-7B        | codellama/CodeLlama-7b-Instruct-hf   | Meta    | coder      | 6.7        | 32     | 4096   | system        |
| CodeLlama-13B       | codellama/CodeLlama-13b-Instruct-hf  | Meta    | coder      | 13.0       | 40     | 5120   | system        |
| CodeLlama-34B       | codellama/CodeLlama-34b-Instruct-hf  | Meta    | coder      | 33.7       | 48     | 8192   | system        |
| Llama-3.1-8B        | meta-llama/Llama-3.1-8B-Instruct     | Meta    | instruct   | 8.0        | 32     | 4096   | system        |
| StarCoder2-15B      | bigcode/starcoder2-15b-instruct-v0.1 | BigCode | coder      | 15.5       | 40     | 6144   | merged        |
| Gemma-3-12B         | google/gemma-3-12b-it                | Google  | instruct   | 12.2       | 48     | 3840   | system        |
| CodeGemma-7B        | google/codegemma-7b-it               | Google  | coder      | 8.5        | 28     | 3072   | merged        |
| Granite-3.1-8B      | ibm-granite/granite-3.1-8b-instruct  | IBM     | instruct   | 8.2        | 40     | 4096   | system        |
| Llama-3.1-8B (base) | meta-llama/Llama-3.1-8B              | Meta    | pretrained | 8.0        | 32     | 4096   | plain         |

### 3.2 Dataset

Table 2 summarises the corpus. The split is by program, never by row; three input cases per program are fixed and their gold outputs executed from the clean parent, so the grader is independent of any transform. Coverage differs by condition because structural transforms decline some programs by design and the unseen family needs enough encodable sites; every paired contrast is computed on the programs common to the cells it compares, fixed before evaluation.

Table 2. **The dataset.** One Python corpus, split by program so no obfuscated variant of a training program can reach evaluation.

| Property                  | Value                                                                                                                       |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Base programs (Python)    | 2231: apps 1584, cruxeval 543, humaneval 104                                                                                |
| Split                     | by program, seed 17: train 1563 / val 111 / held-out 557                                                                    |
| Input cases per program   | 3, gold output executed from the clean parent                                                                               |
| Program length            | mean 8.4 LOC, max 57                                                                                                        |
| Held-out eval, clean      | 557 programs $\times$ 3 cases = 1671 items                                                                                  |
| Coverage per condition    | Tables 3 and 4; structural transforms decline some programs by design, the unseen family needs $\geq 3$ encodable sites     |
| Training rows per adapter | one condition's train split $\times$ 3 cases (4689); breadth pools the five seen conditions                                 |
| Paired contrasts          | on the programs common to every cell in the contrast, fixed before evaluation; program-clustered bootstrap, 2,000 resamples |
| JavaScript                | 168 held-out programs (504 items), transferred intact; not regenerable on the cluster                                       |
| Grading                   | strict normalised exact match, no containment; the backward task executes the produced call against the clean parent        |

### 3.3 Obfuscation taxonomy

Single transforms (Table 3) fall into three families: identifier (L1b, L1r, L2), structural (S1, S2, and S2's two halves S3 and S4) and encoding (X1 and its halves). Each is applied to the clean parent and never stacked during training. H1 is the quarantined held-out obfuscator; its evaluation budget was spent under a pre-registered two-pass discipline, and every unseen-family number in this paper is measured on its trainable sibling X1, which reproduces H1's difficulty at  $r = 0.9992$  (mean absolute difference 0.48 points). Stacks (Table 4) are a separate namespace built by applying parts to the parent in order; order matters, and no system is trained on a stack. Ten stacks contain only seen transforms (six at depth two, four at depth three to four); six contain the unseen family or one of its halves and are the stimulus RQ4 turns on.

### 3.4 Systems

Every system is a LoRA on the same base model, evaluated on still-obfuscated code. *Clean LoRA* (tuned\_L0) is tuned on unobfuscated programs and is the control throughout; one *specialist* per seen transform; *breadth* (mono\_all) is one adapter trained on all five seen transforms pooled; *family* (tuned\_X1) is trained on the unseen family's sibling and is the ceiling for that family. *Routing* mixes the specialists with a learned gate (mole\_router), its argmax, a fixed uniform gate or a gate frozen at random initialisation; the last two separate what the mixture is worth from what the routing is worth. *Merging* combines the specialists in weight space (TIES, DARE-TIES, DARE-linear) or anchors the merge on the clean adapter. The *anchored* objective (cons\_lam3) is Eq. 1 with  $\lambda = 3$ , a

Table 3. **Single transforms.** Each is applied to the clean parent and never stacked. *Cap* is the maximum growth factor over the parent above which a program is declined rather than clipped; *programs / items* is held-out coverage. H1 is the quarantined held-out obfuscator whose evaluation budget is spent; X1 is its trainable sibling and carries every unseen-family number.

| Code | Family     | Transform                                                                                                                                  | Trainable | Cap   | Programs / items | Role                 |
|------|------------|--------------------------------------------------------------------------------------------------------------------------------------------|-----------|-------|------------------|----------------------|
| L0   | none       | normalised original: comments and docstrings stripped, 4-space indent                                                                      | yes       | 1.2×  | 557 / 1671       | clean reference      |
| L1b  | identifier | adversarial renaming of all bindings incl. the entry function, from semantic-inversion tables (max→min) and an unrelated domain vocabulary | yes       | 2.0×  | 553 / 1659       | ladder               |
| L1r  | identifier | random hex renaming of all bindings (v_a3f2, f_9c1e)                                                                                       | yes       | 2.0×  | 557 / 1671       | ladder               |
| L2   | identifier | sequential minification of all bindings (a, b, ..., aa) plus type-annotation stripping                                                     | yes       | 1.5×  | 557 / 1671       | ladder               |
| S1   | structural | control-flow flattening of the entry function into a dispatch loop with randomised state ids and shuffled cases                            | yes       | 6.0×  | 416 / 1248       | ladder               |
| S2   | structural | opaque predicates (computed always-true/false guards) plus never-called dead helpers                                                       | yes       | 4.0×  | 556 / 1668       | ladder               |
| S3   | structural | dead-code insertion only: 1–2 never-called module-level helpers (must be ignored)                                                          | yes       | 4.0×  | 557 / 1671       | stacks only          |
| S4   | structural | opaque predicates only: 1–3 computed guards at statement boundaries (must be reasoned about)                                               | yes       | 4.0×  | 556 / 1668       | stacks only          |
| X1   | encoding   | every string literal through an inline XOR decoder, every + − ^ through a type-guarded algebraic identity, int literals expanded           | yes       | 12.0× | 405 / 1215       | <b>unseen family</b> |
| X1m  | encoding   | X1’s arithmetic half only (mechanisms: [mba])                                                                                              | yes       | 12.0× | 351 / 1053       | mechanism ablation   |
| X1s  | encoding   | X1’s string half only (mechanisms: [str]); site bar 1                                                                                      | yes       | 12.0× | 246 / 738        | mechanism ablation   |
| H1   | heldout    | string encoding (base64 / RC4 string array) plus guarded mixed-boolean-arithmetic rewriting                                                | never     | 12.0× | –                | quarantined          |

plateau from a sweep on the trainable grid: the teacher is a frozen clean-code-tuned adapter reading the unobfuscated parent, the student never sees the parent, and the objective trains on exactly the rows breadth trains on (26,841 rows, 1,257 steps for both); at  $\lambda = 0$  it reduces to breadth identically, which makes breadth the data-matched control. A one-shot in-context baseline completes the set.

### 3.5 Measurement

Forward accuracy is strict normalised exact match on the produced output, with no containment matching; the backward task is graded by executing the produced call against the clean parent. Every contrast is a paired difference on the program subset common to its cells, with a program-clustered bootstrap of 2,000 resamples; bold intervals exclude zero. Decision rules were pre-registered before

Table 4. **Stacked composites.** A separate namespace from the ladder: parts are applied to the clean parent left to right, and order matters because composition does not commute. Size caps were calibrated on a 40-program build at cap 60 ( $p95 \times 1.25$ ) so that the cap is never the binding constraint. No system is ever trained on a stack; the breadth and family adapters see single transforms only.

| Code            | Parts, in order | Depth | X1 inside | Cap   | Prog./items | Group          | Purpose                                                   |
|-----------------|-----------------|-------|-----------|-------|-------------|----------------|-----------------------------------------------------------|
| C_L1b_S1        | L1b→S1          | 2     | no        | 8.0×  | 418 / 1254  | seen d2        | does L1b's special-ist advantage survive stacking?        |
| C_L1r_S1        | L1r→S1          | 2     | no        | 8.0×  | 418 / 1254  | seen d2        | headline identifier ⊗ structural stack                    |
| C_S1_L1r        | S1→L1r          | 2     | no        | 8.0×  | 418 / 1254  | seen d2        | order control for C_L1r_S1 (renames S1's state variables) |
| C_L2_S4         | L2→S4           | 2     | no        | 5.0×  | 556 / 1668  | seen d2        | identifier ⊗ the reason-about-it half of S2               |
| C_L1r_S3        | L1r→S3          | 2     | no        | 5.0×  | 557 / 1671  | seen d2        | identifier ⊗ the ignore-it half of S2                     |
| C_S4_S3         | S4→S3           | 2     | no        | 5.0×  | 556 / 1668  | seen d2        | positive control: re-constructs S2 by composition         |
| C3_L1r_S3_S4    | L1r→S3→S4       | 3     | no        | 10.0× | 556 / 1668  | seen d3–4      | depth 3, identifier first                                 |
| C3_S1_S3_S4     | S1→S3→S4        | 3     | no        | 14.0× | 395 / 1185  | seen d3–4      | depth 3, structural only                                  |
| C3_L1r_S1_S4    | L1r→S1→S4       | 3     | no        | 14.0× | 418 / 1254  | seen d3–4      | depth 3, identifier first, flattening inside              |
| C4_L1r_S1_S3_S4 | L1r→S1→S3→S4    | 4     | no        | 18.0× | 418 / 1254  | seen d3–4      | depth 4                                                   |
| C_L1r_X1        | L1r→X1          | 2     | yes       | 10.0× | 405 / 1215  | unseen d2      | unseen family last, after renaming                        |
| C_X1_S1         | X1→S1           | 2     | yes       | 12.0× | 321 / 963   | unseen d2      | unseen material it-self flattened                         |
| C_S2_X1         | S2→X1           | 2     | yes       | 22.0× | 555 / 1665  | unseen d2      | unseen family applied to dead code too                    |
| C_L1r_X1m       | L1r→X1m         | 2     | yes       | 10.0× | 351 / 1053  | half-unseen d2 | half an unseen family (arithmetic)                        |
| C_S1_X1s        | S1→X1s          | 2     | yes       | 14.0× | 418 / 1254  | half-unseen d2 | half an unseen family (strings)                           |
| C3_L1r_S1_X1    | L1r→S1→X1       | 3     | yes       | 16.0× | 417 / 1251  | unseen d3      | one unseen component at depth 3                           |

submission and are reported by their verdicts even when inconclusive. A cell whose format-failure rate exceeds 0.25 measures the prompt contract rather than the task and enters no mean; untuned format failure measures adherence to that contract, not capability, so screening base models by it is unsound.

#### 4 Every method against every obfuscation

Table 5 is the reference the research questions draw on: the raw accuracy of every method on every condition and stack for CodeLlama-7B, the one model on which every arm exists, forward and, for the ladder, backward. Every number is a strict exact-match accuracy on the held-out programs of

that condition; nothing is normalised. The untuned model scores 0.257 on clean code, 0.168–0.207 on the single seen transforms, 0.119 on the unseen family and 0.089–0.186 on the stacks. Every tuned arm lands between 0.35 and 0.43 on the single seen transforms and their stacks; the columns separate on the unseen family (0.231 breadth, 0.269 clean LoRA, 0.286 anchored, 0.316 family) and on the stacks that contain it. The backward block is read on this model only, because the one-shot inverse template collapses the format on most other panel models; the per-model grids for the whole panel, and the routing and merging arms on every model that has them, are in Appendices A and B.

Table 5. **Every method against every obfuscation, CodeLlama-7B.** Raw accuracy (strict exact match) of each system on each condition and stack, forward (output prediction) and, for the ladder, backward (input prediction). *untuned* is the base model; *ICL* its one-shot prompt; *clean LoRA* is tuned on unobfuscated code; *breadth* on all five seen transforms; *anchored* is the paired-consistency objective; *family* is trained on the unseen family’s sibling; *mixture* is the learned-gate mixture of per-transform specialists; *merge* is the DARE-TIES merge of the specialists (system keys, in order: base, base\_1shot, tuned\_L0, mono\_all, cons\_lam3, tuned\_X1, mole\_router, merge\_dare\_ties). †: format-failure rate above 0.25, the cell measures the prompt contract rather than the task. ‘–’: not run.

| Condition                                              | untuned | ICL    | clean LoRA | breadth | anchored | family | mixture | merge |
|--------------------------------------------------------|---------|--------|------------|---------|----------|--------|---------|-------|
| <i>Forward: output prediction</i>                      |         |        |            |         |          |        |         |       |
| L0                                                     | 0.257   | 0.302  | 0.427      | 0.413   | 0.422    | 0.413  | 0.429   | 0.422 |
| L1b                                                    | 0.198   | 0.255  | 0.358      | 0.388   | 0.402    | 0.346  | 0.381   | 0.370 |
| L1r                                                    | 0.207   | 0.265  | 0.378      | 0.387   | 0.395    | 0.360  | 0.383   | 0.381 |
| L2                                                     | 0.202   | 0.272  | 0.382      | 0.378   | 0.398    | 0.354  | 0.386   | 0.375 |
| S1                                                     | 0.168   | 0.216  | 0.379      | 0.389   | 0.400    | 0.370  | 0.395   | 0.374 |
| S2                                                     | 0.193   | 0.215† | 0.389      | 0.403   | 0.415    | 0.383  | 0.421   | 0.396 |
| X1                                                     | 0.119   | –      | 0.269      | 0.231   | 0.286    | 0.316  | –       | –     |
| C_L1b_S1                                               | 0.141   | –      | 0.279      | 0.345   | 0.360    | –      | 0.325   | 0.298 |
| C_L1r_S1                                               | 0.131   | –      | 0.294      | 0.350   | 0.356    | –      | 0.346   | 0.301 |
| C_S1_L1r                                               | 0.135   | –      | 0.285      | 0.326   | 0.349    | –      | 0.318   | 0.302 |
| C_L2_S4                                                | 0.176   | –      | 0.355      | 0.385   | 0.392    | –      | 0.383   | 0.367 |
| C_L1r_S3                                               | 0.172   | –      | 0.360      | 0.380   | 0.387    | –      | 0.391   | 0.367 |
| C_S4_S3                                                | 0.186   | –      | 0.392      | 0.403   | 0.422    | –      | 0.415   | 0.395 |
| C3_L1r_S3_S4                                           | 0.150   | –      | 0.340      | 0.377   | 0.380    | –      | 0.375   | 0.353 |
| C3_S1_S3_S4                                            | 0.128   | –      | 0.354      | 0.369   | 0.385    | –      | 0.377   | 0.340 |
| C3_L1r_S1_S4                                           | 0.126   | –      | 0.289      | 0.353   | 0.354    | –      | 0.325   | 0.298 |
| C4_L1r_S1_S3_S4                                        | 0.089   | –      | 0.278      | 0.336   | 0.356    | –      | 0.334   | 0.291 |
| C_L1r_X1                                               | 0.102   | –      | 0.184      | 0.199   | 0.222    | 0.254  | 0.189   | 0.192 |
| C_X1_S1                                                | 0.104   | –      | 0.274      | 0.230   | 0.267    | 0.326  | 0.291   | 0.287 |
| C_S2_X1                                                | 0.145   | –      | 0.313      | 0.285   | 0.332    | 0.347  | 0.326   | 0.316 |
| C_L1r_X1m                                              | 0.112   | –      | 0.220      | 0.232   | 0.249    | 0.281  | 0.227   | 0.212 |
| C_S1_X1s                                               | 0.151   | –      | 0.334      | 0.302   | 0.355    | 0.341  | 0.350   | 0.336 |
| C3_L1r_S1_X1                                           | 0.104   | –      | 0.219      | 0.222   | 0.250    | 0.271  | 0.242   | 0.234 |
| <i>Backward: input prediction, graded by execution</i> |         |        |            |         |          |        |         |       |
| L0                                                     | 0.285   | –      | 0.324      | 0.276   | 0.297    | 0.312  | –       | –     |
| L1b                                                    | 0.265   | –      | 0.251      | 0.279   | 0.294    | 0.265  | –       | –     |
| L1r                                                    | 0.315   | –      | 0.327      | 0.276   | 0.293    | 0.327  | –       | –     |
| L2                                                     | 0.282   | –      | 0.324      | 0.276   | 0.275    | 0.324  | –       | –     |
| S1                                                     | 0.261   | –      | 0.188†     | 0.262   | 0.280    | 0.280  | –       | –     |
| S2                                                     | 0.330   | –      | 0.209†     | 0.294   | 0.332    | 0.330  | –       | –     |
| X1                                                     | 0.257   | –      | 0.221†     | 0.237   | 0.281    | 0.291  | –       | –     |

5 RQ1: Do current adaptation methods generalise, or memorise?

Adapting a code model to obfuscated input works, and works easily: tuning on output prediction over obfuscated programs gains roughly **19 points** over the untuned model on the transforms it was trained on. We apply the two tests of Section 3 to the field’s existing answers. These are baselines, not proposals. Merging and breadth fail both tests; routing tracks breadth wherever the two are measured together and inherits its verdict.

Routing composes exactly as far as breadth does, and merging does not compose at all (Table 23, Appendix C). On the ten stacks of seen transforms, a learned router beats a *random* gate by +1.34 [+0.48, +2.17]: the mixture is worth the whole gain (+19.79 over the untuned model) and the routing about a point of it. A hard argmax router is indistinguishable from the soft one (+0.03 [−0.13, +0.19]), so a stacked input gives the mixture no blending advantage, and the router matches breadth training, −0.28 [−1.57, +1.09]. The best merge (DARE-TIES) matches the clean-code control, +0.52 [−0.15, +1.20], the others fall 1.7 to 20.6 points below it, while breadth stands +3.67 [+2.02, +5.24] above the same control.

Table 24 (Appendix C) reports the same merge arms measured across the panel.

Breadth is the interesting case. It composes on stacks built from transforms it saw, and the advantage evaporates the moment a stack contains a family it did not. Table 6 states this as a *difference*, because one interval being positive while another is negative is not a test of the difference between them: breadth’s advantage over clean-code tuning is **2.7 to 5.3 points larger** on seen stacks than on stacks containing an unseen family, and the gap is significant on **every one of the six models** we could measure it on, across four lineages. What breadth buys is recombination of what the training data covered, bounded by that coverage.

Table 6. **RQ1: the dissociation, measured within model on one program set.** Breadth (mono\_all) against the clean-code control (tuned\_L0) on stacks built from *seen* transforms and on stacks containing an *unseen* family. The *difference* is the measured quantity; the two levels are context, and on most models the unseen-side interval straddles zero on its own. Both halves are paired on the 319 programs common to every cell of both groups. Bold intervals exclude zero.

| Model                     | Lineage | Seen stacks          | Unseen inside        | Difference           |
|---------------------------|---------|----------------------|----------------------|----------------------|
| CodeLlama-7B              | Meta    | +3.36 [+1.57, +5.27] | −1.92 [−3.70, −0.14] | +5.28 [+3.20, +7.54] |
| CodeLlama-34B             | Meta    | +2.27 [+0.44, +4.09] | −0.77 [−2.62, +1.12] | +3.03 [+0.78, +5.28] |
| Llama-3.1-8B              | Meta    | +2.30 [+0.47, +4.15] | −1.05 [−2.72, +0.63] | +3.35 [+1.50, +5.30] |
| StarCoder2-15B            | BigCode | +1.50 [−0.02, +3.00] | −1.19 [−2.93, +0.49] | +2.68 [+0.77, +4.65] |
| Gemma-3-12B               | Google  | +2.63 [+0.56, +4.64] | −2.09 [−3.81, −0.42] | +4.72 [+2.62, +6.85] |
| Granite-3.1-8B            | IBM     | +3.66 [+1.54, +5.68] | −0.24 [−2.02, +1.57] | +3.91 [+1.41, +6.32] |
| difference significant on |         |                      |                      | 6 of 6 models        |

The reversal test is independent and gives the same answer. Table 25 (Appendix C) reports the *direction ratio* — backward gain divided by forward gain, both against the untuned model — for the same adapters on the same programs asked backwards. **Every method that fits the forward task directly is negative:** the format-only control at −0.30, clean-code tuning at −0.08, breadth at −0.05; these arms bought forward accuracy by losing backward competence on the very same programs. Both tests point at one explanation: what is acquired is competence *at this task on these surfaces*. Section 6 establishes what that surface is.

Takeaway: RQ1

Routing, merging and breadth are baselines, and none survives leaving its training distribution. Breadth’s advantage over clean-code tuning is 2.7 to 5.3 points larger on stacks of seen transforms than on stacks containing an unseen family, significant on six of six models across four lineages; and every method that fits the forward task has a negative direction ratio, buying forward accuracy with backward competence on the same programs.

6 RQ2: What is memorised?

Three instruments, sharing no machinery, locate what adaptation latches onto.

**The gain tracks identifier surface.** Breadth’s advantage on a stack depends on the stack still containing an identifier transform – renaming, which leaves token-level residue – rather than on depth or the number of mechanisms (Table 7). The rule defining the split was fixed before the unseen-containing stimulus existed, so its second row is an out-of-sample test: the difference is +3.39 on all-seen stacks and grows to +5.85 on stacks containing the unseen family. On the inputs furthest from training, breadth helps only where an identifier surface remains to key on.

Table 7. **RQ2: breadth’s stack gain depends on an identifier transform being present.** Breadth over the clean-code control, split by whether the stack contains an identifier transform, with the *difference* between the groups. The rule defining the split was fixed before the unseen-containing stimulus existed, which makes the second row an out-of-sample test of it. We report the difference because one interval excluding zero while another does not is not a test of the difference between them.

| Stimulus                                            | With identifier      | Structural only      | Difference           |
|-----------------------------------------------------|----------------------|----------------------|----------------------|
| All-seen depth-3/4 stacks (in-sample)               | +5.00 [+3.02, +6.88] | +1.61 [−1.02, +4.23] | +3.39 [+0.82, +6.16] |
| Stacks containing the unseen family (out-of-sample) | +1.98 [−0.12, +4.18] | −3.88 [−6.08, −1.71] | +5.85 [+3.10, +8.72] |

**The unit of transfer is a surface, not a mechanism.** Splitting the held-out family into its two mechanisms, arithmetic rewriting and string encoding, the halves **do not transfer to each other** (+1.49 [−0.95, +3.93] and −0.10 [−2.38, +2.19]), yet **either half alone recovers about 90 %** of the whole adapter’s gain on the full family (+4.12 and +4.20 against +4.61). Composable skills would deliver roughly half each; each delivers nearly all. The test that would discriminate a shared-surface reading from a weakest-link alternative returned UNDECIDED on all three of its pre-registered rules, so shared surface is named as the leading hypothesis and no more.

**Adaptation polarises rather than deepens.** Breadth’s log-probability of the gold answer sits about 4 nats below the clean-code control on every condition, and the whole deficit lives in the items it gets wrong: right, it is more confident (−0.16 versus −0.33); wrong, it is roughly twice as far from the gold (−12.95 versus −7.00). Breadth does not become a worse reader; it becomes a more committed guesser, which is free on seen conditions and expensive on a family where its decisions are more often wrong.

All three agree: adaptation anchors on the surface a transform leaves behind, which is exactly what an obfuscator is free to change. The remedy follows: anchor the model to something the transform *cannot* change, its own behaviour on the unobfuscated parent of the same program, which exists for every training row by construction.

Takeaway: RQ2

Adaptation anchors on the surface a transform leaves behind. Breadth’s gain tracks identifier surface (+3.39 on seen stacks, +5.85 on unseen-containing ones); the two halves of the unseen family do not transfer to each other yet each alone recovers about 90 % of the whole adapter’s gain; and breadth becomes a more committed guesser rather than a better reader. That surface is exactly what an obfuscator is free to change, so the remedy is to anchor to what it cannot: the model’s behaviour on the unobfuscated parent.

7 RQ3: Anchoring to clean-code behaviour

The anchored objective (Section 3, Eq. 1) supervises the student on obfuscated input against a frozen clean-code-tuned teacher reading the unobfuscated parent, on exactly the rows breadth trains on.

**The ablation is the contribution.** Replacing the clean-code-tuned teacher with an untuned base model makes the arm **collapse** (−8.98 [−11.44, −6.43] against breadth on the unseen family); replacing it with the breadth model makes the arm **inherit breadth’s tax** (−4.20 [−6.27, −2.14] against the clean-code teacher) while matching on seen conditions. Across all three teachers the student lands at roughly the teacher’s unseen-family level plus one point. The seen-condition gain comes from the supervised term and is teacher-independent; the unseen-family behaviour is *distilled from the teacher*. The method is “distil from a clean-code-tuned model on the clean parent”, and we name it as such.

Table 8. **RQ3: the three findings across eight models and four lineages**, all read from one evaluation phase. ✓ replicated, ○ inconclusive, × refuted. R1 is breadth’s unseen-family tax (mono\_all − tuned\_L0 on the held-out family); R2 is anchoring removing it (cons\_lam3 − mono\_all); R3 is anchoring’s clean-code cost (cons\_lam3 − tuned\_L0 on clean code). Bold intervals exclude zero.

| Model                               | Lineage | R1 unseen tax          | R2 anchoring repair    | R3 clean-code cost     |
|-------------------------------------|---------|------------------------|------------------------|------------------------|
| CodeLlama-7B                        | Meta    | −3.71 [−5.93, −1.56] ✓ | +5.44 [+3.37, +7.41] ✓ | −0.54 [−1.98, +1.08] ✓ |
| CodeLlama-13B                       | Meta    | −4.04 [−6.26, −1.89] ✓ | +3.87 [+2.06, +5.77] ✓ | −0.72 [−2.28, +0.90] ✓ |
| CodeLlama-34B                       | Meta    | −2.72 [−5.02, −0.49] ✓ | +3.62 [+1.40, +5.76] ✓ | +0.48 [−0.84, +1.80] ✓ |
| Llama-3.1-8B                        | Meta    | −2.72 [−4.70, −0.91] ✓ | +3.38 [+1.57, +5.36] ✓ | −0.30 [−1.98, +1.26] ✓ |
| StarCoder2-15B                      | BigCode | −2.72 [−4.94, −0.58] ✓ | +2.31 [+0.58, +4.03] ✓ | −0.96 [−2.51, +0.60] ✓ |
| Gemma-3-12B                         | Google  | −4.70 [−6.93, −2.55] ✓ | +2.64 [+0.91, +4.53] ✓ | −1.80 [−3.53, −0.06] × |
| CodeGemma-7B                        | Google  | −1.98 [−3.95, +0.16] ○ | −0.41 [−2.39, +1.49] ○ | −2.04 [−3.77, −0.18] × |
| Granite-3.1-8B                      | IBM     | −0.91 [−2.97, +1.15] ○ | −1.89 [−3.54, −0.25] × | −3.41 [−5.39, −1.26] × |
| replicated / inconclusive / refuted |         | 6 / 2 / 0              | 6 / 1 / 1              | 5 / 0 / 3              |

**What it repairs, and on which models.** Table 8 reads all eight models from one evaluation phase; every previously published contrast recomputed through it falls inside the new interval (largest disagreement 0.85 points), so a departure is a property of the model. Breadth’s unseen-family tax has the same sign on every model, six significantly, across four lineages and 7B–34B. The repair holds on six of eight models and **reverses on Granite-3.1-8B**, where the anchored arm is worse than breadth on the unseen family. “Anchoring pays no clean-code cost” holds on five of eight. Every failure falls outside the lineage the findings were discovered on, where all three are four-of-four; with  $n = 8$  and vendor confounded with lineage (Fisher’s exact  $p = 0.0714$  one-sided) that is a warning that single-lineage evaluation would have hidden all of it, not a claim about lineages.

### Takeaway: RQ3

Distilling from a clean-code-*tuned* teacher on the unobfuscated parent, on exactly the rows breadth trains on, removes breadth's unseen-family tax on six of eight models and reverses it on one (Granite-3.1-8B), at no clean-code cost on five of eight. The teacher ablation shows where the effect lives: the seen-condition gain is teacher-independent, the unseen-family behaviour is the teacher's.

## 8 RQ4: Does the anchored model generalise?

If the diagnosis is right, anchoring to something the transform cannot change should move both tests specifically where the input has left the training distribution.

Table 9. **RQ4: the divergence ladder.** Composites that contain a transform family the model has never seen. Breadth's advantage flips sign the moment such a component enters, while anchoring rises *above* the clean-code control. The pre-registered rule required both that anchoring hold and that breadth fall below the control; only the first is established, since the pooled interval at d2 straddles zero. CodeLlama-7B, 287 programs common to all six composites, fixed before any system was evaluated.

| Level | Stimulus                      | mono-clean           | cons-clean           | cons-mono            |
|-------|-------------------------------|----------------------|----------------------|----------------------|
| d0    | depth 2, all seen             | +3.49 [+2.04, +5.01] | –                    | +0.87 [–0.59, +2.33] |
| d1    | depth 3–4, all seen           | +4.15 [+2.37, +5.91] | +5.02 [+3.41, +6.69] | +0.87 [–0.59, +2.33] |
| d2    | depth 2, <b>unseen inside</b> | –1.71 [–3.60, +0.31] | +2.13 [+0.66, +3.60] | +3.84 [+1.90, +5.70] |
| d3    | depth 3, <b>unseen inside</b> | –0.35 [–2.67, +1.86] | +3.26 [+0.81, +5.92] | +3.60 [+1.40, +5.81] |

**Composition.** Table 9 is the divergence ladder, from all-seen depth-2 stacks (d0) to stacks containing a family the model has never seen (d2, d3); the d2 and d3 stimulus did not exist before this work. Once an unseen component enters, breadth's advantage over the clean-code control **flips sign**, from +3.49 and +4.15 to –1.71 and –0.35, and anchoring, which merely matches breadth on seen stacks (+0.87 [–0.59, +2.33]), rises **above the control** (+2.13, +3.26) and **above breadth** (+3.84, +3.60). The pre-registered rule required both that anchoring hold and that breadth fall below the control; only the first is established, because the pooled d2 interval spans zero, so by that rule the outcome is **UNDECIDED**. Split by RQ2's identifier rule, committed before the first d2 cell was written, identifier-containing stacks sit at +1.98 [–0.12, +4.18] and structural-containing stacks at –3.88 [–6.08, –1.71]: breadth is broken where the stack leaves it no identifier surface to key on. Across the panel (Table 26, Appendix C) the anchoring advantage on unseen-containing stacks replicates on five of eight models, is inconclusive on three and refuted on none; family exposure is significant on all eight and is the largest effect in the table.

**Reversal.** In Table 25, anchoring's direction ratio is +0.03: the only arm in the supervised-fine-tuning family whose forward gain cost nothing backwards. It does **not** acquire backward competence (+0.62 [–1.56, +2.81] against the untuned model); the claim is that every existing method pays backward competence for its forward gain and anchoring is the first that does not. The one arm that gains in both directions is trained on the unseen family's sibling (+1.63 [+0.11, +3.20] backwards): exposure to a family transfers in a way exposure to transforms does not.

**The bound.** On the unseen family alone the anchored model is as robust as clean-code tuning and no more (–0.30, –0.33, +0.74 against the control at 7B, 13B, 34B; seventh of thirty-three arms), and every arm that beats it there was trained on the family's sibling. Anchoring buys breadth's recombination benefit at little cost, extends it to composed inputs the model has never seen, and

stops paying the backward price — and it does not add the capability breadth lacks. It is a repair of what adaptation breaks, not a route to invariance.

Takeaway: RQ4

The anchored model’s advantage appears only where the input diverges from training: level with breadth on seen stacks, above the clean-code control and above breadth once an unseen family enters (five of eight models replicate, none refute; the strict pre-registered rule is undecided), and a direction ratio of +0.03 where every other supervised arm is negative. On the unseen family alone it is as robust as clean-code tuning and no more: a repair of what adaptation breaks, not a route to invariance.

9 Related Work

This paper joins three distinct conversations in the literature. First, we differentiate from the deobfuscation (DOBF) lineage by strictly evaluating code that remains obfuscated, measuring semantic competence under transformation rather than recovery capability. Second, in the domain of model merging and modular adaptation, we contribute a negative result supported by a highly rigorous random-gate control, warning that task-vector geometry is often dominated by initialisation rather than genuine knowledge. Finally, we contribute to the growing literature on robustness to semantics-preserving transformations and direction specificity in code understanding.

10 Threats to Validity

**The held-out family is a proxy.** Every unseen-family number is measured on X1, a trainable sibling of the quarantined family H1, whose budget was spent under a pre-registered two-pass discipline; X1 reproduces H1’s difficulty at  $r = 0.9992$  and transfer between them was lossless (0.08 points) for arms trained on the sibling. It is a single family pair; a second hard family with a different surface would settle whether “family” means “surface family”. **Coverage is not uniform.** Conditions cover different program sets; every contrast runs on a subset recorded before evaluation, and where the intersection moves an estimate we report both. **Eight models is a small sample for a claim about models**, and Section 7 declines to make one; the panel’s strongest non-lineage model is also the one an untuned capability screen would have discarded. **Coverage of the baselines.** Merging is measured forward only, on four models (Table 24); routing is measured on seen stacks on one model, so its verdict rests on matching breadth where the two are measured together; the direction ratio is one model deep, because the one-shot inverse template collapses the format on most other panel models. **Instrument failures are reported, not omitted.** The causal attention instrument is inert (an identifier knockout moves the gold log-probability by under 0.5 % everywhere), so the attention account is correlational; the weakest-link test in Section 6 returned undecided. **Single language.** Every headline is Python; the JavaScript corpus exists but cannot be regenerated on our cluster.

11 Data Availability

Data/code is available in project’s website [].

## References

- [1] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating Large Language Models Trained on Code. arXiv:2107.03374 [cs.LG] <https://arxiv.org/abs/2107.03374>
- [2] Christian Collberg, Clark Thomborson, and Douglas Low. 1997. A Taxonomy of Obfuscating Transformations. *Technical Report* (1997).
- [3] Christian Collberg, Clark Thomborson, and Douglas Low. 1998. Manufacturing cheap, resilient, and stealthy opaque constructs. In *Proceedings of the 25th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages* (San Diego, California, USA) (POPL '98). Association for Computing Machinery, New York, NY, USA, 184–196. doi:10.1145/268946.268962
- [4] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, Trevor Cohn, Yulan He, and Yang Liu (Eds.). Association for Computational Linguistics, Online, 1536–1547. doi:10.18653/v1/2020.findings-emnlp.139
- [5] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie LIU, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, Michele Tufano, Shao Kun Deng, Colin Clement, Dawn Drain, Neel Sundaresan, Jian Yin, Daxin Jiang, and Ming Zhou. 2021. GraphCode(BERT): Pre-training Code Representations with Data Flow. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=jLoC4ez43PZ>
- [6] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. 2022. Competition-level code generation with AlphaCode. *Science* 378, 6624 (Dec. 2022), 1092–1097. doi:10.1126/science.abq1158
- [7] Anh H. N. Nguyen, Jack Le, Ilse Lahnstein Coronado, and Tien N. Nguyen. 2026. The Effect of Code Obfuscation on Human Program Comprehension. arXiv:2603.07668 [cs.SE] <https://arxiv.org/abs/2603.07668>
- [8] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2023. CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis. arXiv:2203.13474 [cs.LG] <https://arxiv.org/abs/2203.13474>
- [9] Serge Lionel Nikiema, Jordan Samhi, Abdoul Kader Kaboré, Jacques Klein, and Tegawendé F. Bissyandé. 2025. The Code Barrier: What LLMs Actually Understand? arXiv:2504.10557 [cs.SE] <https://arxiv.org/abs/2504.10557>
- [10] Philip O'Kane, Sakir Sezer, and Kieran McLaughlin. 2011. Obfuscation: The Hidden Malware. *IEEE Security and Privacy* 9, 5 (2011), 41–47. doi:10.1109/MSP.2011.98
- [11] Daniel Votipka, Seth Rabin, Kristopher Micinski, Jeffrey S. Foster, and Michelle L. Mazurek. 2019. An Observational Investigation of Reverse Engineers' Process and Mental Models. In *Extended Abstracts of the 2019 CHI Conference on Human Factors in Computing Systems* (Glasgow, Scotland Uk) (CHI EA '19). Association for Computing Machinery, New York, NY, USA, 1–6. doi:10.1145/3290607.3313040

**A Per-model master grids**

One table per panel model: every condition and every stack, forward and backward, with the raw accuracy, the change against the untuned model, and the percentage of the untuned model’s clean-code accuracy. Cells marked *fmt* exceed the pre-registered format-failure gate (0.25) and enter no mean; the gate is applied per cell, which is stricter than the pooled registration. The untuned CodeLlama-34B sits at 0.254 on clean code, a hair over the bar, which gates its percentage column while its deltas stand. Rows marked ‘–’ are cells not yet run at the time of generation.

**B Routing and merging grids**

Every routing and merging system on every condition and stack that has it, for the models on which those arms exist (routing: CodeLlama-7B; merging: CodeLlama-7B, Granite-3.1-8B, Llama-3.1-8B, StarCoder2-15B), with the same three numbers as Appendix A. The routing arms are MoLE mixtures of the eight per-transform experts; *uniform* and *random* gates separate the mixture’s worth from the routing’s. Every routing cell ran through the mixture engine.

**C Supplementary tables**

The contrast tables referenced from the research questions that are not the one table each question carries in the body: routing and merging on stacks (Table 23), merging across the panel (Table 24), the direction ratio (Table 25), the anchoring advantage by model (Table 26), and the cross-model summary in relative terms (Table 27).

Table 10. **Master grid, CodeLlama-7B**. Every condition and every stack, forward (output prediction) and backward (input prediction, ladder only). Per system: *acc* raw accuracy;  $\Delta$  points against the untuned model on the same condition; % accuracy as a percentage of the untuned model’s clean-code accuracy in the same direction, i.e. how much of the original accuracy the system brings back (100 = the untuned model on clean code). *fmt*: format-failure rate above 0.25, the cell measures the prompt contract rather than the task and enters no mean. Pooled rows average the stacks above them. ‘-’: cell not run.

| Condition                                              | untuned |     | ICL        |          |     | clean LoRA |          |     | breadth  |          |     | anchored  |          |     | family   |          |     |
|--------------------------------------------------------|---------|-----|------------|----------|-----|------------|----------|-----|----------|----------|-----|-----------|----------|-----|----------|----------|-----|
|                                                        | base    |     | base_1shot |          |     | tuned_L0   |          |     | mono_all |          |     | cons_lam3 |          |     | tuned_X1 |          |     |
|                                                        | acc     | %   | acc        | $\Delta$ | %   | acc        | $\Delta$ | %   | acc      | $\Delta$ | %   | acc       | $\Delta$ | %   | acc      | $\Delta$ | %   |
| <i>Forward: output prediction</i>                      |         |     |            |          |     |            |          |     |          |          |     |           |          |     |          |          |     |
| L0                                                     | 0.257   | 100 | 0.302      | +4.5     | 117 | 0.427      | +17.0    | 166 | 0.413    | +15.6    | 161 | 0.422     | +16.5    | 164 | 0.413    | +15.6    | 161 |
| L1b                                                    | 0.198   | 77  | 0.255      | +5.7     | 99  | 0.358      | +16.0    | 139 | 0.388    | +19.0    | 151 | 0.402     | +20.4    | 156 | 0.346    | +14.8    | 135 |
| L1r                                                    | 0.207   | 81  | 0.265      | +5.8     | 103 | 0.378      | +17.1    | 147 | 0.387    | +18.0    | 151 | 0.395     | +18.8    | 154 | 0.360    | +15.3    | 140 |
| L2                                                     | 0.202   | 79  | 0.272      | +7.0     | 106 | 0.382      | +18.0    | 149 | 0.378    | +17.7    | 147 | 0.398     | +19.6    | 155 | 0.354    | +15.2    | 138 |
| S1                                                     | 0.168   | 66  | 0.216      | +4.7     | 84  | 0.379      | +21.0    | 147 | 0.389    | +22.1    | 151 | 0.400     | +23.2    | 156 | 0.370    | +20.2    | 144 |
| S2                                                     | 0.193   | 75  | fmt        | fmt      | fmt | 0.389      | +19.6    | 152 | 0.403    | +20.9    | 157 | 0.415     | +22.2    | 162 | 0.383    | +19.0    | 149 |
| X1                                                     | 0.119   | 46  | -          | -        | -   | 0.269      | +14.9    | 105 | 0.231    | +11.2    | 90  | 0.286     | +16.6    | 111 | 0.316    | +19.7    | 123 |
| C_L1b_S1                                               | 0.141   | 55  | -          | -        | -   | 0.279      | +13.7    | 108 | 0.345    | +20.4    | 134 | 0.360     | +21.9    | 140 | -        | -        | -   |
| C_L1r_S1                                               | 0.131   | 51  | -          | -        | -   | 0.294      | +16.3    | 114 | 0.350    | +21.9    | 136 | 0.356     | +22.5    | 139 | -        | -        | -   |
| C_S1_L1r                                               | 0.135   | 53  | -          | -        | -   | 0.285      | +15.0    | 111 | 0.326    | +19.2    | 127 | 0.349     | +21.4    | 136 | -        | -        | -   |
| C_L2_S4                                                | 0.176   | 68  | -          | -        | -   | 0.355      | +17.9    | 138 | 0.385    | +20.9    | 150 | 0.392     | +21.7    | 153 | -        | -        | -   |
| C_L1r_S3                                               | 0.172   | 67  | -          | -        | -   | 0.360      | +18.8    | 140 | 0.380    | +20.8    | 148 | 0.387     | +21.5    | 151 | -        | -        | -   |
| C_S4_S3                                                | 0.186   | 72  | -          | -        | -   | 0.392      | +20.6    | 153 | 0.403    | +21.7    | 157 | 0.422     | +23.6    | 164 | -        | -        | -   |
| <i>seen stacks, pooled (6)</i>                         | 0.157   | 61  | -          | -        | -   | 0.327      | +17.1    | 127 | 0.365    | +20.8    | 142 | 0.378     | +22.1    | 147 | -        | -        | -   |
| C3_L1r_S3_S4                                           | 0.150   | 58  | -          | -        | -   | 0.340      | +19.0    | 132 | 0.377    | +22.7    | 147 | 0.380     | +23.0    | 148 | -        | -        | -   |
| C3_S1_S3_S4                                            | 0.128   | 50  | -          | -        | -   | 0.354      | +22.6    | 138 | 0.369    | +24.1    | 144 | 0.385     | +25.7    | 150 | -        | -        | -   |
| C3_L1r_S1_S4                                           | 0.126   | 49  | -          | -        | -   | 0.289      | +16.3    | 112 | 0.353    | +22.7    | 137 | 0.354     | +22.7    | 138 | -        | -        | -   |
| C4_L1r_S1_S3_S4                                        | 0.089   | 35  | -          | -        | -   | 0.278      | +18.8    | 108 | 0.336    | +24.7    | 131 | 0.356     | +26.7    | 139 | -        | -        | -   |
| <i>depth-3/4 stacks, pooled (4)</i>                    | 0.123   | 48  | -          | -        | -   | 0.315      | +19.2    | 123 | 0.359    | +23.5    | 140 | 0.369     | +24.5    | 144 | -        | -        | -   |
| C_L1r_X1                                               | 0.102   | 40  | -          | -        | -   | 0.184      | +8.2     | 72  | 0.199    | +9.6     | 77  | 0.222     | +12.0    | 87  | 0.254    | +15.2    | 99  |
| C_X1_S1                                                | 0.104   | 40  | -          | -        | -   | 0.274      | +17.0    | 107 | 0.230    | +12.6    | 89  | 0.267     | +16.3    | 104 | 0.326    | +22.2    | 127 |
| C_S2_X1                                                | 0.145   | 57  | -          | -        | -   | 0.313      | +16.8    | 122 | 0.285    | +13.9    | 111 | 0.332     | +18.6    | 129 | 0.347    | +20.2    | 135 |
| <i>unseen-containing stacks, pooled (3)</i>            | 0.117   | 46  | -          | -        | -   | 0.257      | +14.0    | 100 | 0.238    | +12.1    | 93  | 0.274     | +15.7    | 107 | 0.309    | +19.2    | 120 |
| C_L1r_X1m                                              | 0.112   | 44  | -          | -        | -   | 0.220      | +10.7    | 85  | 0.232    | +12.0    | 90  | 0.249     | +13.7    | 97  | 0.281    | +16.9    | 110 |
| C_S1_X1s                                               | 0.151   | 59  | -          | -        | -   | 0.334      | +18.3    | 130 | 0.302    | +15.1    | 117 | 0.355     | +20.4    | 138 | 0.341    | +19.0    | 133 |
| C3_L1r_S1_X1                                           | 0.104   | 40  | -          | -        | -   | 0.219      | +11.5    | 85  | 0.222    | +11.8    | 86  | 0.250     | +14.6    | 97  | 0.271    | +16.7    | 106 |
| <i>Backward: input prediction, graded by execution</i> |         |     |            |          |     |            |          |     |          |          |     |           |          |     |          |          |     |
| L0                                                     | 0.285   | 100 | -          | -        | -   | 0.324      | +3.9     | 114 | 0.276    | -0.8     | 97  | 0.297     | +1.3     | 104 | 0.312    | +2.7     | 109 |
| L1b                                                    | 0.265   | 93  | -          | -        | -   | 0.251      | -1.3     | 88  | 0.279    | +1.4     | 98  | 0.294     | +2.9     | 103 | 0.265    | +0.1     | 93  |
| L1r                                                    | 0.315   | 111 | -          | -        | -   | 0.327      | +1.1     | 115 | 0.276    | -3.9     | 97  | 0.293     | -2.3     | 103 | 0.327    | +1.1     | 115 |
| L2                                                     | 0.282   | 99  | -          | -        | -   | 0.324      | +4.2     | 114 | 0.276    | -0.6     | 97  | 0.275     | -0.7     | 97  | 0.324    | +4.2     | 114 |
| S1                                                     | 0.261   | 92  | -          | -        | -   | fmt        | fmt      | fmt | 0.262    | +0.1     | 92  | 0.280     | +1.9     | 98  | 0.280    | +1.8     | 98  |
| S2                                                     | 0.330   | 116 | -          | -        | -   | fmt        | fmt      | fmt | 0.294    | -3.6     | 103 | 0.332     | +0.2     | 117 | 0.330    | +0.0     | 116 |
| X1                                                     | 0.257   | 90  | -          | -        | -   | fmt        | fmt      | fmt | 0.237    | -2.0     | 83  | 0.281     | +2.5     | 99  | 0.291    | +3.4     | 102 |

Table 11. **Master grid, CodeLlama-13B**. Every condition and every stack, forward (output prediction) and backward (input prediction, ladder only). Per system: *acc* raw accuracy;  $\Delta$  points against the untuned model on the same condition; % accuracy as a percentage of the untuned model’s clean-code accuracy in the same direction, i.e. how much of the original accuracy the system brings back (100 = the untuned model on clean code). *fmt*: format-failure rate above 0.25, the cell measures the prompt contract rather than the task and enters no mean. Pooled rows average the stacks above them. ‘-’: cell not run.

| Condition                                              | untuned |     | ICL   |          |     | clean LoRA |          |     | breadth  |          |     | anchored  |          |     | family   |          |     |
|--------------------------------------------------------|---------|-----|-------|----------|-----|------------|----------|-----|----------|----------|-----|-----------|----------|-----|----------|----------|-----|
|                                                        | base    |     | base  | 1shot    |     | tuned_L0   |          |     | mono_all |          |     | cons_lam3 |          |     | tuned_X1 |          |     |
|                                                        | acc     | %   | acc   | $\Delta$ | %   | acc        | $\Delta$ | %   | acc      | $\Delta$ | %   | acc       | $\Delta$ | %   | acc      | $\Delta$ | %   |
| <i>Forward: output prediction</i>                      |         |     |       |          |     |            |          |     |          |          |     |           |          |     |          |          |     |
| L0                                                     | 0.252   | 100 | fmt   | fmt      | fmt | 0.469      | +21.7    | 186 | 0.446    | +19.3    | 177 | 0.462     | +21.0    | 183 | 0.463    | +21.1    | 184 |
| L1b                                                    | 0.222   | 88  | fmt   | fmt      | fmt | 0.397      | +17.6    | 158 | 0.423    | +20.1    | 168 | 0.438     | +21.7    | 174 | 0.379    | +15.7    | 150 |
| L1r                                                    | 0.225   | 89  | fmt   | fmt      | fmt | 0.415      | +19.0    | 165 | 0.420    | +19.5    | 167 | 0.439     | +21.4    | 174 | 0.407    | +18.1    | 161 |
| L2                                                     | 0.228   | 90  | fmt   | fmt      | fmt | 0.407      | +17.8    | 161 | 0.417    | +18.9    | 166 | 0.432     | +20.4    | 171 | 0.401    | +17.3    | 159 |
| S1                                                     | 0.213   | 85  | 0.216 | +0.2     | 86  | 0.408      | +19.5    | 162 | 0.413    | +20.0    | 164 | 0.435     | +22.1    | 172 | 0.410    | +19.6    | 163 |
| S2                                                     | 0.184   | 73  | fmt   | fmt      | fmt | 0.422      | +23.8    | 168 | 0.445    | +26.0    | 176 | 0.469     | +28.4    | 186 | 0.422    | +23.8    | 168 |
| X1                                                     | 0.149   | 59  | -     | -        | -   | 0.294      | +14.5    | 117 | 0.254    | +10.5    | 101 | 0.292     | +14.3    | 116 | 0.353    | +20.4    | 140 |
| C_L1b_S1                                               | 0.183   | 72  | -     | -        | -   | 0.318      | +13.6    | 126 | 0.367    | +18.4    | 146 | 0.375     | +19.2    | 149 | 0.320    | +13.7    | 127 |
| C_L1r_S1                                               | 0.175   | 69  | -     | -        | -   | 0.318      | +14.4    | 126 | 0.371    | +19.6    | 147 | 0.376     | +20.1    | 149 | 0.342    | +16.8    | 136 |
| C_S1_L1r                                               | 0.172   | 68  | -     | -        | -   | 0.315      | +14.4    | 125 | 0.359    | +18.8    | 142 | 0.363     | +19.2    | 144 | 0.350    | +17.8    | 139 |
| C_L2_S4                                                | 0.200   | 79  | -     | -        | -   | 0.389      | +19.0    | 154 | 0.422    | +22.2    | 167 | 0.432     | +23.2    | 171 | 0.385    | +18.5    | 153 |
| C_L1r_S3                                               | 0.186   | 74  | -     | -        | -   | 0.407      | +22.0    | 161 | 0.409    | +22.3    | 162 | 0.434     | +24.8    | 172 | 0.399    | +21.3    | 158 |
| C_S4_S3                                                | 0.192   | 76  | -     | -        | -   | 0.424      | +23.2    | 168 | 0.436    | +24.4    | 173 | 0.459     | +26.7    | 182 | 0.433    | +24.1    | 172 |
| seen stacks, pooled (6)                                | 0.185   | 73  | -     | -        | -   | 0.362      | +17.8    | 144 | 0.394    | +21.0    | 156 | 0.407     | +22.2    | 161 | 0.372    | +18.7    | 147 |
| C3_L1r_S3_S4                                           | -       | -   | -     | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| C3_S1_S3_S4                                            | -       | -   | -     | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| C3_L1r_S1_S4                                           | -       | -   | -     | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| C4_L1r_S1_S3_S4                                        | -       | -   | -     | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| depth-3/4 stacks, pooled (4)                           | -       | -   | -     | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| C_L1r_X1                                               | 0.123   | 49  | -     | -        | -   | 0.199      | +7.7     | 79  | 0.213    | +9.0     | 84  | 0.236     | +11.3    | 93  | 0.284    | +16.1    | 113 |
| C_X1_S1                                                | 0.138   | 55  | -     | -        | -   | 0.295      | +15.7    | 117 | 0.270    | +13.2    | 107 | 0.307     | +16.8    | 122 | 0.341    | +20.3    | 135 |
| C_S2_X1                                                | 0.160   | 64  | -     | -        | -   | 0.335      | +17.4    | 133 | 0.334    | +17.4    | 133 | 0.356     | +19.6    | 141 | 0.376    | +21.6    | 149 |
| unseen-containing stacks, pooled (3)                   | 0.140   | 56  | -     | -        | -   | 0.276      | +13.6    | 110 | 0.272    | +13.2    | 108 | 0.300     | +15.9    | 119 | 0.334    | +19.3    | 132 |
| C_L1r_X1m                                              | 0.138   | 55  | -     | -        | -   | 0.246      | +10.8    | 98  | 0.248    | +11.0    | 98  | 0.279     | +14.1    | 110 | 0.319    | +18.2    | 127 |
| C_S1_X1s                                               | 0.179   | 71  | -     | -        | -   | 0.364      | +18.5    | 144 | 0.345    | +16.6    | 137 | 0.360     | +18.1    | 143 | 0.368    | +18.9    | 146 |
| C3_L1r_S1_X1                                           | 0.128   | 51  | -     | -        | -   | 0.233      | +10.5    | 92  | 0.234    | +10.6    | 93  | 0.249     | +12.1    | 99  | 0.291    | +16.3    | 116 |
| <i>Backward: input prediction, graded by execution</i> |         |     |       |          |     |            |          |     |          |          |     |           |          |     |          |          |     |
| L0                                                     | 0.418   | 100 | -     | -        | -   | 0.405      | -1.3     | 97  | fmt      | fmt      | fmt | fmt       | fmt      | fmt | fmt      | fmt      | fmt |
| L1b                                                    | 0.385   | 92  | -     | -        | -   | 0.391      | +0.6     | 94  | fmt      | fmt      | fmt | fmt       | fmt      | fmt | fmt      | fmt      | fmt |
| L1r                                                    | 0.433   | 104 | -     | -        | -   | 0.417      | -1.6     | 100 | fmt      | fmt      | fmt | fmt       | fmt      | fmt | fmt      | fmt      | fmt |
| L2                                                     | 0.396   | 95  | -     | -        | -   | 0.390      | -0.6     | 93  | fmt      | fmt      | fmt | fmt       | fmt      | fmt | fmt      | fmt      | fmt |
| S1                                                     | 0.408   | 97  | -     | -        | -   | 0.424      | +1.6     | 101 | fmt      | fmt      | fmt | fmt       | fmt      | fmt | fmt      | fmt      | fmt |
| S2                                                     | 0.303   | 72  | -     | -        | -   | 0.394      | +9.2     | 94  | fmt      | fmt      | fmt | fmt       | fmt      | fmt | fmt      | fmt      | fmt |
| X1                                                     | 0.264   | 63  | -     | -        | -   | 0.267      | +0.3     | 64  | fmt      | fmt      | fmt | fmt       | fmt      | fmt | fmt      | fmt      | fmt |

Table 12. **Master grid, CodeLlama-34B.** Every condition and every stack, forward (output prediction) and backward (input prediction, ladder only). Per system: *acc* raw accuracy;  $\Delta$  points against the untuned model on the same condition; % accuracy as a percentage of the untuned model’s clean-code accuracy in the same direction, i.e. how much of the original accuracy the system brings back (100 = the untuned model on clean code). *fmt*: format-failure rate above 0.25, the cell measures the prompt contract rather than the task and enters no mean. Pooled rows average the stacks above them. ‘-’: cell not run.

| Condition                                              | untuned |     | ICL        |          |     | clean LoRA |          |     | breadth  |          |     | anchored  |          |     | family   |          |     |
|--------------------------------------------------------|---------|-----|------------|----------|-----|------------|----------|-----|----------|----------|-----|-----------|----------|-----|----------|----------|-----|
|                                                        | base    |     | base_1shot |          |     | tuned_L0   |          |     | mono_all |          |     | cons_lam3 |          |     | tuned_X1 |          |     |
|                                                        | acc     | %   | acc        | $\Delta$ | %   | acc        | $\Delta$ | %   | acc      | $\Delta$ | %   | acc       | $\Delta$ | %   | acc      | $\Delta$ | %   |
| <i>Forward: output prediction</i>                      |         |     |            |          |     |            |          |     |          |          |     |           |          |     |          |          |     |
| L0                                                     |         |     |            |          |     | 0.520      |          |     | 0.494    |          |     | 0.525     |          |     | 0.502    |          |     |
| L1b                                                    | 0.205   | fnt | fnt        | fnt      | fnt | 0.417      | +21.2    | fnt | 0.469    | +26.4    | fnt | 0.494     | +28.9    | fnt | 0.401    | +19.6    | fnt |
| L1r                                                    | 0.220   | fnt | fnt        | fnt      | fnt | 0.469      | +25.0    | fnt | 0.457    | +23.8    | fnt | 0.487     | +26.8    | fnt | 0.459    | +24.0    | fnt |
| L2                                                     | 0.231   | fnt | fnt        | fnt      | fnt | 0.466      | +23.5    | fnt | 0.465    | +23.5    | fnt | 0.478     | +24.7    | fnt | 0.456    | +22.6    | fnt |
| S1                                                     | 0.222   | fnt | fnt        | fnt      | fnt | 0.472      | +24.9    | fnt | 0.467    | +24.5    | fnt | 0.496     | +27.3    | fnt | 0.461    | +23.9    | fnt |
| S2                                                     | 0.191   | fnt | fnt        | fnt      | fnt | 0.484      | +29.3    | fnt | 0.484    | +29.3    | fnt | 0.525     | +33.5    | fnt | 0.478    | +28.7    | fnt |
| X1                                                     | 0.135   | fnt | -          | -        | -   | 0.325      | +19.0    | fnt | 0.298    | +16.3    | fnt | 0.334     | +19.9    | fnt | 0.381    | +24.5    | fnt |
| C_L1b_S1                                               | 0.156   | fnt | -          | -        | -   | 0.322      | +16.6    | fnt | 0.399    | +24.3    | fnt | 0.421     | +26.5    | fnt | 0.336    | +18.0    | fnt |
| C_L1r_S1                                               | 0.184   | fnt | -          | -        | -   | 0.351      | +16.8    | fnt | 0.393    | +21.0    | fnt | 0.427     | +24.3    | fnt | 0.366    | +18.2    | fnt |
| C_S1_L1r                                               | 0.180   | fnt | -          | -        | -   | 0.367      | +18.8    | fnt | 0.405    | +22.5    | fnt | 0.425     | +24.6    | fnt | 0.379    | +20.0    | fnt |
| C_L2_S4                                                | 0.191   | fnt | -          | -        | -   | 0.431      | +24.1    | fnt | 0.454    | +26.3    | fnt | 0.478     | +28.7    | fnt | 0.431    | +24.1    | fnt |
| C_L1r_S3                                               | 0.169   | fnt | -          | -        | -   | 0.451      | +28.1    | fnt | 0.454    | +28.5    | fnt | 0.482     | +31.3    | fnt | 0.437    | +26.8    | fnt |
| C_S4_S3                                                | 0.199   | fnt | -          | -        | -   | 0.482      | +28.3    | fnt | 0.487    | +28.7    | fnt | 0.521     | +32.2    | fnt | 0.476    | +27.7    | fnt |
| <i>seen stacks, pooled (6)</i>                         | 0.180   | fnt | -          | -        | -   | 0.401      | +22.1    | fnt | 0.432    | +25.2    | fnt | 0.459     | +27.9    | fnt | 0.404    | +22.4    | fnt |
| C3_L1r_S3_S4                                           | -       | -   | -          | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| C3_S1_S3_S4                                            | -       | -   | -          | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| C3_L1r_S1_S4                                           | -       | -   | -          | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| C4_L1r_S1_S3_S4                                        | -       | -   | -          | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| <i>depth-3/4 stacks, pooled (4)</i>                    | -       | -   | -          | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| C_L1r_X1                                               | 0.115   | fnt | -          | -        | -   | 0.226      | +11.0    | fnt | 0.249    | +13.3    | fnt | 0.278     | +16.2    | fnt | 0.275    | +16.0    | fnt |
| C_X1_S1                                                | 0.125   | fnt | -          | -        | -   | 0.317      | +19.2    | fnt | 0.290    | +16.5    | fnt | 0.339     | +21.4    | fnt | 0.372    | +24.7    | fnt |
| C_S2_X1                                                | 0.176   | fnt | -          | -        | -   | 0.372      | +19.6    | fnt | 0.355    | +17.9    | fnt | 0.403     | +22.7    | fnt | 0.395    | +21.9    | fnt |
| <i>unseen-containing stacks, pooled (3)</i>            | 0.139   | fnt | -          | -        | -   | 0.305      | +16.6    | fnt | 0.298    | +15.9    | fnt | 0.340     | +20.1    | fnt | 0.348    | +20.9    | fnt |
| C_L1r_X1m                                              | 0.128   | fnt | -          | -        | -   | 0.261      | +13.3    | fnt | 0.275    | +14.6    | fnt | 0.294     | +16.5    | fnt | 0.317    | +18.9    | fnt |
| C_S1_X1s                                               | 0.176   | fnt | -          | -        | -   | 0.390      | +21.4    | fnt | 0.382    | +20.6    | fnt | 0.415     | +23.9    | fnt | 0.411    | +23.5    | fnt |
| C3_L1r_S1_X1                                           | 0.118   | fnt | -          | -        | -   | 0.256      | +13.8    | fnt | 0.262    | +14.4    | fnt | 0.297     | +17.9    | fnt | 0.293    | +17.5    | fnt |
| <i>Backward: input prediction, graded by execution</i> |         |     |            |          |     |            |          |     |          |          |     |           |          |     |          |          |     |
| L0                                                     | 0.466   | 100 | -          | -        | -   | fnt        | fnt      | fnt | fnt      | fnt      | fnt | fnt       | fnt      | fnt | 0.326    | -14.1    | 70  |
| L1b                                                    | 0.381   | 82  | -          | -        | -   | fnt        | fnt      | fnt | fnt      | fnt      | fnt | fnt       | fnt      | fnt | fnt      | fnt      | fnt |
| L1r                                                    | 0.414   | 89  | -          | -        | -   | fnt        | fnt      | fnt | fnt      | fnt      | fnt | fnt       | fnt      | fnt | fnt      | fnt      | fnt |
| L2                                                     | 0.417   | 89  | -          | -        | -   | 0.371      | -4.5     | 80  | 0.296    | -12.0    | 64  | fnt       | fnt      | fnt | 0.314    | -10.3    | 67  |
| S1                                                     | 0.405   | 87  | -          | -        | -   | fnt        | fnt      | fnt | fnt      | fnt      | fnt | fnt       | fnt      | fnt | 0.319    | -8.7     | 68  |
| S2                                                     | 0.434   | 93  | -          | -        | -   | fnt        | fnt      | fnt | 0.321    | -11.3    | 69  | fnt       | fnt      | fnt | 0.311    | -12.4    | 67  |
| X1                                                     | 0.320   | 69  | -          | -        | -   | fnt        | fnt      | fnt | fnt      | fnt      | fnt | fnt       | fnt      | fnt | fnt      | fnt      | fnt |

Table 13. **Master grid, Llama-3.1-8B**. Every condition and every stack, forward (output prediction) and backward (input prediction, ladder only). Per system: *acc* raw accuracy;  $\Delta$  points against the untuned model on the same condition; % accuracy as a percentage of the untuned model’s clean-code accuracy in the same direction, i.e. how much of the original accuracy the system brings back (100 = the untuned model on clean code). *fmt*: format-failure rate above 0.25, the cell measures the prompt contract rather than the task and enters no mean. Pooled rows average the stacks above them. ‘-’: cell not run.

| Condition                                              | untuned |     | ICL        |          |     | clean LoRA |          |     | breadth  |          |     | anchored  |          |     | family   |          |     |
|--------------------------------------------------------|---------|-----|------------|----------|-----|------------|----------|-----|----------|----------|-----|-----------|----------|-----|----------|----------|-----|
|                                                        | base    |     | base_1shot |          |     | tuned_L0   |          |     | mono_all |          |     | cons_lam3 |          |     | tuned_X1 |          |     |
|                                                        | acc     | %   | acc        | $\Delta$ | %   | acc        | $\Delta$ | %   | acc      | $\Delta$ | %   | acc       | $\Delta$ | %   | acc      | $\Delta$ | %   |
| <i>Forward: output prediction</i>                      |         |     |            |          |     |            |          |     |          |          |     |           |          |     |          |          |     |
| L0                                                     | 0.258   | 100 | 0.320      | +6.2     | 124 | 0.448      | +19.0    | 174 | 0.426    | +16.8    | 165 | 0.445     | +18.7    | 172 | 0.459    | +20.1    | 178 |
| L1b                                                    | 0.222   | 86  | 0.276      | +5.4     | 107 | 0.361      | +13.9    | 140 | 0.387    | +16.5    | 150 | 0.407     | +18.5    | 158 | 0.369    | +14.7    | 143 |
| L1r                                                    | 0.242   | 94  | 0.284      | +4.3     | 110 | 0.387      | +14.6    | 150 | 0.387    | +14.5    | 150 | 0.405     | +16.3    | 157 | 0.378    | +13.7    | 147 |
| L2                                                     | 0.215   | 83  | 0.278      | +6.3     | 108 | 0.391      | +17.6    | 152 | 0.386    | +17.1    | 150 | 0.409     | +19.4    | 158 | 0.387    | +17.2    | 150 |
| S1                                                     | 0.213   | 83  | 0.256      | +4.3     | 99  | 0.395      | +18.2    | 153 | 0.391    | +17.7    | 151 | 0.413     | +20.0    | 160 | 0.409    | +19.6    | 158 |
| S2                                                     | 0.205   | 79  | 0.272      | +6.7     | 106 | 0.413      | +20.8    | 160 | 0.413    | +20.8    | 160 | 0.445     | +23.9    | 172 | 0.438    | +23.3    | 170 |
| X1                                                     | 0.132   | 51  | -          | -        | -   | 0.249      | +11.7    | 96  | 0.222    | +9.0     | 86  | 0.255     | +12.4    | 99  | 0.332    | +20.0    | 129 |
| C_L1b_S1                                               | 0.152   | 59  | -          | -        | -   | 0.279      | +12.7    | 108 | 0.338    | +18.7    | 131 | 0.368     | +21.6    | 143 | 0.305    | +15.3    | 118 |
| C_L1r_S1                                               | 0.162   | 63  | -          | -        | -   | 0.303      | +14.1    | 118 | 0.352    | +19.0    | 136 | 0.363     | +20.1    | 141 | 0.314    | +15.2    | 122 |
| C_S1_L1r                                               | 0.159   | 62  | -          | -        | -   | 0.277      | +11.8    | 107 | 0.338    | +17.9    | 131 | 0.354     | +19.5    | 137 | 0.305    | +14.6    | 118 |
| C_L2_S4                                                | 0.205   | 79  | -          | -        | -   | 0.349      | +14.3    | 135 | 0.386    | +18.1    | 149 | 0.401     | +19.6    | 155 | 0.350    | +14.5    | 136 |
| C_L1r_S3                                               | 0.198   | 77  | -          | -        | -   | 0.367      | +16.9    | 142 | 0.382    | +18.4    | 148 | 0.398     | +19.9    | 154 | 0.369    | +17.1    | 143 |
| C_S4_S3                                                | 0.214   | 83  | -          | -        | -   | 0.421      | +20.7    | 163 | 0.411    | +19.7    | 159 | 0.444     | +23.0    | 172 | 0.431    | +21.8    | 167 |
| seen stacks, pooled (6)                                | 0.182   | 70  | -          | -        | -   | 0.332      | +15.1    | 129 | 0.368    | +18.6    | 142 | 0.388     | +20.6    | 150 | 0.346    | +16.4    | 134 |
| C3_L1r_S3_S4                                           | -       | -   | -          | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| C3_S1_S3_S4                                            | -       | -   | -          | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| C3_L1r_S1_S4                                           | -       | -   | -          | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| C4_L1r_S1_S3_S4                                        | -       | -   | -          | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| depth-3/4 stacks, pooled (4)                           | -       | -   | -          | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| C_L1r_X1                                               | 0.100   | 39  | -          | -        | -   | 0.165      | +6.4     | 64  | 0.181    | +8.1     | 70  | 0.207     | +10.6    | 80  | 0.257    | +15.7    | 100 |
| C_X1_S1                                                | 0.130   | 50  | -          | -        | -   | 0.271      | +14.1    | 105 | 0.225    | +9.5     | 87  | 0.253     | +12.3    | 98  | 0.300    | +17.0    | 116 |
| C_S2_X1                                                | 0.170   | 66  | -          | -        | -   | 0.296      | +12.6    | 115 | 0.284    | +11.4    | 110 | 0.317     | +14.7    | 123 | 0.360    | +19.0    | 139 |
| unseen-containing stacks, pooled (3)                   | 0.134   | 52  | -          | -        | -   | 0.244      | +11.1    | 95  | 0.230    | +9.6     | 89  | 0.259     | +12.5    | 100 | 0.306    | +17.2    | 118 |
| C_L1r_X1m                                              | 0.115   | 45  | -          | -        | -   | 0.184      | +6.9     | 71  | 0.216    | +10.1    | 84  | 0.232     | +11.7    | 90  | 0.291    | +17.6    | 113 |
| C_S1_X1s                                               | 0.193   | 75  | -          | -        | -   | 0.341      | +14.8    | 132 | 0.299    | +10.6    | 116 | 0.335     | +14.2    | 130 | 0.355    | +16.2    | 138 |
| C3_L1r_S1_X1                                           | 0.105   | 41  | -          | -        | -   | 0.194      | +8.9     | 75  | 0.207    | +10.2    | 80  | 0.224     | +11.9    | 87  | 0.251    | +14.6    | 97  |
| <i>Backward: input prediction, graded by execution</i> |         |     |            |          |     |            |          |     |          |          |     |           |          |     |          |          |     |
| L0                                                     | 0.424   | 100 | -          | -        | -   | 0.393      | -3.2     | 93  | fmt      | fmt      | fmt | fmt       | fmt      | fmt | 0.408    | -1.7     | 96  |
| L1b                                                    | 0.322   | 76  | -          | -        | -   | 0.330      | +0.7     | 78  | fmt      | fmt      | fmt | fmt       | fmt      | fmt | 0.334    | +1.1     | 79  |
| L1r                                                    | 0.369   | 87  | -          | -        | -   | 0.385      | +1.6     | 91  | fmt      | fmt      | fmt | fmt       | fmt      | fmt | 0.375    | +0.6     | 88  |
| L2                                                     | 0.363   | 86  | -          | -        | -   | 0.387      | +2.3     | 91  | fmt      | fmt      | fmt | fmt       | fmt      | fmt | 0.358    | -0.5     | 84  |
| S1                                                     | 0.348   | 82  | -          | -        | -   | 0.402      | +5.4     | 95  | fmt      | fmt      | fmt | fmt       | fmt      | fmt | 0.401    | +5.3     | 94  |
| S2                                                     | 0.368   | 87  | -          | -        | -   | 0.390      | +2.2     | 92  | fmt      | fmt      | fmt | fmt       | fmt      | fmt | 0.409    | +4.1     | 96  |
| X1                                                     | 0.279   | 66  | -          | -        | -   | 0.310      | +3.1     | 73  | fmt      | fmt      | fmt | fmt       | fmt      | fmt | 0.343    | +6.4     | 81  |

Table 14. **Master grid, StarCoder2-15B.** Every condition and every stack, forward (output prediction) and backward (input prediction, ladder only). Per system: *acc* raw accuracy;  $\Delta$  points against the untuned model on the same condition; % accuracy as a percentage of the untuned model’s clean-code accuracy in the same direction, i.e. how much of the original accuracy the system brings back (100 = the untuned model on clean code). *fmt*: format-failure rate above 0.25, the cell measures the prompt contract rather than the task and enters no mean. Pooled rows average the stacks above them. ‘-’: cell not run.

| Condition                                              | untuned |     | ICL        |          |     | clean LoRA |          |     | breadth  |          |     | anchored  |          |     | family   |          |     |
|--------------------------------------------------------|---------|-----|------------|----------|-----|------------|----------|-----|----------|----------|-----|-----------|----------|-----|----------|----------|-----|
|                                                        | base    |     | base_1shot |          |     | tuned_L0   |          |     | mono_all |          |     | cons_lam3 |          |     | tuned_X1 |          |     |
|                                                        | acc     | %   | acc        | $\Delta$ | %   | acc        | $\Delta$ | %   | acc      | $\Delta$ | %   | acc       | $\Delta$ | %   | acc      | $\Delta$ | %   |
| <i>Forward: output prediction</i>                      |         |     |            |          |     |            |          |     |          |          |     |           |          |     |          |          |     |
| L0                                                     | fmt     | fmt | fmt        | fmt      | fmt | 0.542      | fmt      | fmt | 0.529    | fmt      | fmt | 0.532     | fmt      | fmt | 0.513    | fmt      | fmt |
| L1b                                                    | fmt     | fmt | fmt        | fmt      | fmt | 0.431      | fmt      | fmt | 0.487    | fmt      | fmt | 0.510     | fmt      | fmt | 0.402    | fmt      | fmt |
| L1r                                                    | fmt     | fmt | fmt        | fmt      | fmt | 0.483      | fmt      | fmt | 0.485    | fmt      | fmt | 0.509     | fmt      | fmt | 0.448    | fmt      | fmt |
| L2                                                     | fmt     | fmt | fmt        | fmt      | fmt | 0.479      | fmt      | fmt | 0.486    | fmt      | fmt | 0.503     | fmt      | fmt | 0.437    | fmt      | fmt |
| S1                                                     | fmt     | fmt | fmt        | fmt      | fmt | 0.508      | fmt      | fmt | 0.487    | fmt      | fmt | 0.501     | fmt      | fmt | 0.470    | fmt      | fmt |
| S2                                                     | fmt     | fmt | fmt        | fmt      | fmt | 0.512      | fmt      | fmt | 0.514    | fmt      | fmt | 0.539     | fmt      | fmt | 0.501    | fmt      | fmt |
| X1                                                     | fmt     | fmt | -          | -        | -   | 0.347      | fmt      | fmt | 0.320    | fmt      | fmt | 0.343     | fmt      | fmt | 0.381    | fmt      | fmt |
| C_L1b_S1                                               | fmt     | fmt | -          | -        | -   | 0.360      | fmt      | fmt | 0.416    | fmt      | fmt | 0.456     | fmt      | fmt | 0.336    | fmt      | fmt |
| C_L1r_S1                                               | fmt     | fmt | -          | -        | -   | 0.393      | fmt      | fmt | 0.423    | fmt      | fmt | 0.440     | fmt      | fmt | 0.377    | fmt      | fmt |
| C_S1_L1r                                               | fmt     | fmt | -          | -        | -   | 0.377      | fmt      | fmt | 0.406    | fmt      | fmt | 0.431     | fmt      | fmt | 0.370    | fmt      | fmt |
| C_L2_S4                                                | fmt     | fmt | -          | -        | -   | 0.454      | fmt      | fmt | 0.470    | fmt      | fmt | 0.508     | fmt      | fmt | 0.421    | fmt      | fmt |
| C_L1r_S3                                               | fmt     | fmt | -          | -        | -   | 0.473      | fmt      | fmt | 0.483    | fmt      | fmt | 0.504     | fmt      | fmt | 0.435    | fmt      | fmt |
| C_S4_S3                                                | fmt     | fmt | -          | -        | -   | 0.511      | fmt      | fmt | 0.516    | fmt      | fmt | 0.539     | fmt      | fmt | 0.490    | fmt      | fmt |
| <i>seen stacks, pooled (6)</i>                         | fmt     | fmt | -          | -        | -   | 0.428      | fmt      | fmt | 0.452    | fmt      | fmt | 0.480     | fmt      | fmt | 0.405    | fmt      | fmt |
| C3_L1r_S3_S4                                           | -       | -   | -          | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| C3_S1_S3_S4                                            | -       | -   | -          | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| C3_L1r_S1_S4                                           | -       | -   | -          | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| C4_L1r_S1_S3_S4                                        | -       | -   | -          | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| <i>depth-3/4 stacks, pooled (4)</i>                    | -       | -   | -          | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| C_L1r_X1                                               | fmt     | fmt | -          | -        | -   | 0.248      | fmt      | fmt | 0.274    | fmt      | fmt | 0.286     | fmt      | fmt | 0.288    | fmt      | fmt |
| C_X1_S1                                                | fmt     | fmt | -          | -        | -   | 0.357      | fmt      | fmt | 0.322    | fmt      | fmt | 0.342     | fmt      | fmt | 0.401    | fmt      | fmt |
| C_S2_X1                                                | fmt     | fmt | -          | -        | -   | 0.403      | fmt      | fmt | 0.375    | fmt      | fmt | 0.397     | fmt      | fmt | 0.420    | fmt      | fmt |
| <i>unseen-containing stacks, pooled (3)</i>            | fmt     | fmt | -          | -        | -   | 0.336      | fmt      | fmt | 0.324    | fmt      | fmt | 0.341     | fmt      | fmt | 0.370    | fmt      | fmt |
| C_L1r_X1m                                              | fmt     | fmt | -          | -        | -   | 0.307      | fmt      | fmt | 0.323    | fmt      | fmt | 0.338     | fmt      | fmt | 0.342    | fmt      | fmt |
| C_S1_X1s                                               | fmt     | fmt | -          | -        | -   | 0.434      | fmt      | fmt | 0.408    | fmt      | fmt | 0.434     | fmt      | fmt | 0.435    | fmt      | fmt |
| C3_L1r_S1_X1                                           | fmt     | fmt | -          | -        | -   | 0.278      | fmt      | fmt | 0.298    | fmt      | fmt | 0.302     | fmt      | fmt | 0.306    | fmt      | fmt |
| <i>Backward: input prediction, graded by execution</i> |         |     |            |          |     |            |          |     |          |          |     |           |          |     |          |          |     |
| L0                                                     | fmt     | fmt | -          | -        | -   | fmt        | fmt      | fmt | fmt      | fmt      | fmt | fmt       | fmt      | fmt | fmt      | fmt      | fmt |
| L1b                                                    | fmt     | fmt | -          | -        | -   | fmt        | fmt      | fmt | fmt      | fmt      | fmt | fmt       | fmt      | fmt | fmt      | fmt      | fmt |
| L1r                                                    | fmt     | fmt | -          | -        | -   | fmt        | fmt      | fmt | fmt      | fmt      | fmt | fmt       | fmt      | fmt | fmt      | fmt      | fmt |
| L2                                                     | fmt     | fmt | -          | -        | -   | fmt        | fmt      | fmt | fmt      | fmt      | fmt | fmt       | fmt      | fmt | fmt      | fmt      | fmt |
| S1                                                     | fmt     | fmt | -          | -        | -   | fmt        | fmt      | fmt | fmt      | fmt      | fmt | fmt       | fmt      | fmt | fmt      | fmt      | fmt |
| S2                                                     | fmt     | fmt | -          | -        | -   | fmt        | fmt      | fmt | fmt      | fmt      | fmt | fmt       | fmt      | fmt | fmt      | fmt      | fmt |
| X1                                                     | fmt     | fmt | -          | -        | -   | fmt        | fmt      | fmt | fmt      | fmt      | fmt | fmt       | fmt      | fmt | fmt      | fmt      | fmt |

Table 15. **Master grid, Gemma-3-12B**. Every condition and every stack, forward (output prediction) and backward (input prediction, ladder only). Per system: *acc* raw accuracy;  $\Delta$  points against the untuned model on the same condition; % accuracy as a percentage of the untuned model’s clean-code accuracy in the same direction, i.e. how much of the original accuracy the system brings back (100 = the untuned model on clean code). *fmt*: format-failure rate above 0.25, the cell measures the prompt contract rather than the task and enters no mean. Pooled rows average the stacks above them. ‘-’: cell not run.

| Condition                                              | untuned |     | ICL        |          |     | clean LoRA |          |     | breadth  |          |     | anchored  |          |     | family   |          |     |
|--------------------------------------------------------|---------|-----|------------|----------|-----|------------|----------|-----|----------|----------|-----|-----------|----------|-----|----------|----------|-----|
|                                                        | base    |     | base_1shot |          |     | tuned_L0   |          |     | mono_all |          |     | cons_lam3 |          |     | tuned_X1 |          |     |
|                                                        | acc     | %   | acc        | $\Delta$ | %   | acc        | $\Delta$ | %   | acc      | $\Delta$ | %   | acc       | $\Delta$ | %   | acc      | $\Delta$ | %   |
| <i>Forward: output prediction</i>                      |         |     |            |          |     |            |          |     |          |          |     |           |          |     |          |          |     |
| L0                                                     | fmt     | fmt | fmt        | fmt      | fmt | 0.520      | fmt      | fmt | 0.507    | fmt      | fmt | 0.502     | fmt      | fmt | 0.506    | fmt      | fmt |
| L1b                                                    | fmt     | fmt | fmt        | fmt      | fmt | 0.441      | fmt      | fmt | 0.475    | fmt      | fmt | 0.488     | fmt      | fmt | 0.402    | fmt      | fmt |
| L1r                                                    | fmt     | fmt | fmt        | fmt      | fmt | 0.459      | fmt      | fmt | 0.463    | fmt      | fmt | 0.472     | fmt      | fmt | 0.447    | fmt      | fmt |
| L2                                                     | fmt     | fmt | fmt        | fmt      | fmt | 0.456      | fmt      | fmt | 0.471    | fmt      | fmt | 0.478     | fmt      | fmt | 0.441    | fmt      | fmt |
| S1                                                     | fmt     | fmt | fmt        | fmt      | fmt | 0.456      | fmt      | fmt | 0.457    | fmt      | fmt | 0.483     | fmt      | fmt | 0.452    | fmt      | fmt |
| S2                                                     | fmt     | fmt | fmt        | fmt      | fmt | 0.500      | fmt      | fmt | 0.503    | fmt      | fmt | 0.504     | fmt      | fmt | 0.489    | fmt      | fmt |
| X1                                                     | 0.152   | fmt | -          | -        | -   | 0.310      | +15.7    | fmt | 0.263    | +11.0    | fmt | 0.289     | +13.7    | fmt | 0.376    | +22.3    | fmt |
| C_L1b_S1                                               | fmt     | fmt | -          | -        | -   | 0.333      | fmt      | fmt | 0.406    | fmt      | fmt | 0.420     | fmt      | fmt | 0.330    | fmt      | fmt |
| C_L1r_S1                                               | fmt     | fmt | -          | -        | -   | 0.336      | fmt      | fmt | 0.389    | fmt      | fmt | 0.416     | fmt      | fmt | 0.344    | fmt      | fmt |
| C_S1_L1r                                               | fmt     | fmt | -          | -        | -   | 0.334      | fmt      | fmt | 0.393    | fmt      | fmt | 0.414     | fmt      | fmt | 0.329    | fmt      | fmt |
| C_L2_S4                                                | fmt     | fmt | -          | -        | -   | 0.438      | fmt      | fmt | 0.460    | fmt      | fmt | 0.469     | fmt      | fmt | 0.424    | fmt      | fmt |
| C_L1r_S3                                               | fmt     | fmt | -          | -        | -   | 0.438      | fmt      | fmt | 0.462    | fmt      | fmt | 0.469     | fmt      | fmt | 0.423    | fmt      | fmt |
| C_S4_S3                                                | fmt     | fmt | -          | -        | -   | 0.491      | fmt      | fmt | 0.506    | fmt      | fmt | 0.503     | fmt      | fmt | 0.481    | fmt      | fmt |
| seen stacks, pooled (6)                                | fmt     | fmt | -          | -        | -   | 0.395      | fmt      | fmt | 0.436    | fmt      | fmt | 0.448     | fmt      | fmt | 0.388    | fmt      | fmt |
| C3_L1r_S3_S4                                           | -       | -   | -          | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| C3_S1_S3_S4                                            | -       | -   | -          | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| C3_L1r_S1_S4                                           | -       | -   | -          | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| C4_L1r_S1_S3_S4                                        | -       | -   | -          | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| depth-3/4 stacks, pooled (4)                           | -       | -   | -          | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| C_L1r_X1                                               | 0.102   | fmt | -          | -        | -   | 0.195      | +9.3     | fmt | 0.206    | +10.4    | fmt | 0.224     | +12.2    | fmt | 0.285    | +18.3    | fmt |
| C_X1_S1                                                | 0.172   | fmt | -          | -        | -   | 0.317      | +14.6    | fmt | 0.271    | +10.0    | fmt | 0.296     | +12.5    | fmt | 0.358    | +18.6    | fmt |
| C_S2_X1                                                | 0.201   | fmt | -          | -        | -   | 0.368      | +16.6    | fmt | 0.349    | +14.8    | fmt | 0.364     | +16.3    | fmt | 0.412    | +21.1    | fmt |
| unseen-containing stacks, pooled (3)                   | 0.158   | fmt | -          | -        | -   | 0.293      | +13.5    | fmt | 0.275    | +11.7    | fmt | 0.295     | +13.7    | fmt | 0.352    | +19.3    | fmt |
| C_L1r_X1m                                              | fmt     | fmt | -          | -        | -   | 0.220      | fmt      | fmt | 0.235    | fmt      | fmt | 0.244     | fmt      | fmt | 0.312    | fmt      | fmt |
| C_S1_X1s                                               | 0.255   | fmt | -          | -        | -   | 0.395      | +14.0    | fmt | 0.374    | +12.0    | fmt | 0.395     | +14.0    | fmt | 0.401    | +14.7    | fmt |
| C3_L1r_S1_X1                                           | 0.136   | fmt | -          | -        | -   | 0.225      | +8.9     | fmt | 0.236    | +10.0    | fmt | 0.258     | +12.2    | fmt | 0.260    | +12.4    | fmt |
| <i>Backward: input prediction, graded by execution</i> |         |     |            |          |     |            |          |     |          |          |     |           |          |     |          |          |     |
| L0                                                     | 0.491   | 100 | -          | -        | -   | 0.344      | -14.7    | 70  | 0.452    | -3.9     | 92  | fmt       | fmt      | fmt | 0.474    | -1.7     | 97  |
| L1b                                                    | 0.411   | 84  | -          | -        | -   | fmt        | fmt      | fmt | 0.439    | +2.8     | 90  | fmt       | fmt      | fmt | 0.417    | +0.6     | 85  |
| L1r                                                    | 0.440   | 90  | -          | -        | -   | 0.350      | -9.0     | 71  | 0.423    | -1.7     | 86  | fmt       | fmt      | fmt | 0.439    | -0.1     | 90  |
| L2                                                     | 0.409   | 83  | -          | -        | -   | 0.335      | -7.5     | 68  | 0.414    | +0.4     | 84  | fmt       | fmt      | fmt | 0.440    | +3.1     | 90  |
| S1                                                     | 0.417   | 85  | -          | -        | -   | fmt        | fmt      | fmt | 0.445    | +2.7     | 91  | fmt       | fmt      | fmt | 0.446    | +2.8     | 91  |
| S2                                                     | 0.458   | 93  | -          | -        | -   | 0.388      | -7.0     | 79  | 0.463    | +0.5     | 94  | fmt       | fmt      | fmt | 0.472    | +1.4     | 96  |
| X1                                                     | 0.317   | 65  | -          | -        | -   | 0.290      | -2.7     | 59  | 0.340    | +2.3     | 69  | fmt       | fmt      | fmt | 0.374    | +5.7     | 76  |

Table 16. **Master grid, CodeGemma-7B**. Every condition and every stack, forward (output prediction) and backward (input prediction, ladder only). Per system: *acc* raw accuracy;  $\Delta$  points against the untuned model on the same condition; % accuracy as a percentage of the untuned model’s clean-code accuracy in the same direction, i.e. how much of the original accuracy the system brings back (100 = the untuned model on clean code). *fmt*: format-failure rate above 0.25, the cell measures the prompt contract rather than the task and enters no mean. Pooled rows average the stacks above them. ‘-’: cell not run.

| Condition                                              | untuned |     | ICL        |          |     | clean LoRA |          |     | breadth  |          |     | anchored  |          |     | family   |          |     |
|--------------------------------------------------------|---------|-----|------------|----------|-----|------------|----------|-----|----------|----------|-----|-----------|----------|-----|----------|----------|-----|
|                                                        | base    |     | base_1shot |          |     | tuned_L0   |          |     | mono_all |          |     | cons_lam3 |          |     | tuned_X1 |          |     |
|                                                        | acc     | %   | acc        | $\Delta$ | %   | acc        | $\Delta$ | %   | acc      | $\Delta$ | %   | acc       | $\Delta$ | %   | acc      | $\Delta$ | %   |
| <i>Forward: output prediction</i>                      |         |     |            |          |     |            |          |     |          |          |     |           |          |     |          |          |     |
| L0                                                     | 0.237   | 100 | fmt        | fmt      | fmt | 0.465      | +22.9    | 197 | 0.439    | +20.2    | 186 | 0.445     | +20.8    | 188 | 0.462    | +22.6    | 195 |
| L1b                                                    | 0.185   | 78  | fmt        | fmt      | fmt | 0.382      | +19.7    | 161 | 0.425    | +24.0    | 180 | 0.435     | +25.1    | 184 | 0.345    | +16.0    | 146 |
| L1r                                                    | 0.213   | 90  | fmt        | fmt      | fmt | 0.399      | +18.6    | 169 | 0.417    | +20.4    | 176 | 0.417     | +20.4    | 176 | 0.390    | +17.7    | 165 |
| L2                                                     | 0.208   | 88  | fmt        | fmt      | fmt | 0.412      | +20.4    | 174 | 0.422    | +21.4    | 178 | 0.422     | +21.4    | 178 | 0.392    | +18.3    | 166 |
| S1                                                     | 0.195   | 82  | 0.212      | +1.7     | 90  | 0.405      | +21.0    | 171 | 0.415    | +22.1    | 176 | 0.421     | +22.6    | 178 | 0.423    | +22.8    | 179 |
| S2                                                     | 0.198   | 84  | fmt        | fmt      | fmt | 0.450      | +25.2    | 190 | 0.429    | +23.1    | 181 | 0.448     | +25.0    | 189 | 0.444    | +24.6    | 188 |
| X1                                                     | 0.143   | 61  | -          | -        | -   | 0.271      | +12.8    | 115 | 0.251    | +10.8    | 106 | 0.247     | +10.4    | 104 | 0.348    | +20.4    | 147 |
| C_L1b_S1                                               | 0.148   | 63  | -          | -        | -   | 0.286      | +13.7    | 121 | 0.372    | +22.3    | 157 | 0.368     | +21.9    | 156 | 0.287    | +13.9    | 121 |
| C_L1r_S1                                               | 0.144   | 61  | -          | -        | -   | 0.315      | +17.2    | 133 | 0.373    | +22.9    | 158 | 0.367     | +22.3    | 155 | 0.304    | +16.0    | 129 |
| C_S1_L1r                                               | 0.132   | 56  | -          | -        | -   | 0.288      | +15.6    | 122 | 0.363    | +23.1    | 154 | 0.361     | +22.8    | 153 | 0.299    | +16.7    | 127 |
| C_L2_S4                                                | 0.176   | 75  | -          | -        | -   | 0.391      | +21.5    | 165 | 0.423    | +24.7    | 179 | 0.421     | +24.5    | 178 | 0.374    | +19.8    | 158 |
| C_L1r_S3                                               | 0.183   | 77  | -          | -        | -   | 0.396      | +21.3    | 168 | 0.415    | +23.2    | 175 | 0.410     | +22.7    | 173 | 0.375    | +19.2    | 159 |
| C_S4_S3                                                | 0.202   | 85  | -          | -        | -   | 0.448      | +24.7    | 189 | 0.430    | +22.9    | 182 | 0.441     | +23.9    | 186 | 0.446    | +24.5    | 189 |
| <i>seen stacks, pooled (6)</i>                         | 0.164   | 69  | -          | -        | -   | 0.354      | +19.0    | 150 | 0.396    | +23.2    | 167 | 0.395     | +23.0    | 167 | 0.348    | +18.4    | 147 |
| C3_L1r_S3_S4                                           | -       | -   | -          | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| C3_S1_S3_S4                                            | -       | -   | -          | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| C3_L1r_S1_S4                                           | -       | -   | -          | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| C4_L1r_S1_S3_S4                                        | -       | -   | -          | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| <i>depth-3/4 stacks, pooled (4)</i>                    | -       | -   | -          | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| C_L1r_X1                                               | 0.118   | 50  | -          | -        | -   | 0.194      | +7.6     | 82  | 0.212    | +9.4     | 90  | 0.197     | +7.9     | 83  | 0.245    | +12.8    | 104 |
| C_X1_S1                                                | 0.144   | 61  | -          | -        | -   | 0.285      | +14.0    | 120 | 0.261    | +11.6    | 110 | 0.247     | +10.3    | 105 | 0.339    | +19.4    | 143 |
| C_S2_X1                                                | 0.188   | 80  | -          | -        | -   | 0.344      | +15.6    | 146 | 0.323    | +13.5    | 136 | 0.326     | +13.8    | 138 | 0.379    | +19.1    | 160 |
| <i>unseen-containing stacks, pooled (3)</i>            | 0.150   | 63  | -          | -        | -   | 0.274      | +12.4    | 116 | 0.265    | +11.5    | 112 | 0.257     | +10.7    | 109 | 0.321    | +17.1    | 136 |
| C_L1r_X1m                                              | 0.130   | 55  | -          | -        | -   | 0.214      | +8.4     | 90  | 0.256    | +12.5    | 108 | 0.234     | +10.4    | 99  | 0.288    | +15.8    | 122 |
| C_S1_X1s                                               | 0.186   | 79  | -          | -        | -   | 0.338      | +15.2    | 143 | 0.351    | +16.5    | 148 | 0.339     | +15.3    | 143 | 0.386    | +20.0    | 163 |
| C3_L1r_S1_X1                                           | 0.134   | 57  | -          | -        | -   | 0.218      | +8.4     | 92  | 0.241    | +10.6    | 102 | 0.246     | +11.2    | 104 | 0.254    | +11.9    | 107 |
| <i>Backward: input prediction, graded by execution</i> |         |     |            |          |     |            |          |     |          |          |     |           |          |     |          |          |     |
| L0                                                     | 0.289   | 100 | -          | -        | -   | fmt        | fmt      | fmt | fmt      | fmt      | fmt | fmt       | fmt      | fmt | fmt      | fmt      | fmt |
| L1b                                                    | 0.230   | 79  | -          | -        | -   | fmt        | fmt      | fmt | fmt      | fmt      | fmt | fmt       | fmt      | fmt | fmt      | fmt      | fmt |
| L1r                                                    | 0.286   | 99  | -          | -        | -   | fmt        | fmt      | fmt | fmt      | fmt      | fmt | fmt       | fmt      | fmt | fmt      | fmt      | fmt |
| L2                                                     | 0.263   | 91  | -          | -        | -   | fmt        | fmt      | fmt | fmt      | fmt      | fmt | fmt       | fmt      | fmt | fmt      | fmt      | fmt |
| S1                                                     | 0.253   | 88  | -          | -        | -   | fmt        | fmt      | fmt | fmt      | fmt      | fmt | fmt       | fmt      | fmt | fmt      | fmt      | fmt |
| S2                                                     | 0.245   | 85  | -          | -        | -   | fmt        | fmt      | fmt | fmt      | fmt      | fmt | fmt       | fmt      | fmt | fmt      | fmt      | fmt |
| X1                                                     | fmt     | fmt | -          | -        | -   | fmt        | fmt      | fmt | fmt      | fmt      | fmt | fmt       | fmt      | fmt | fmt      | fmt      | fmt |

Table 17. **Master grid, Granite-3.1-8B**. Every condition and every stack, forward (output prediction) and backward (input prediction, ladder only). Per system: *acc* raw accuracy;  $\Delta$  points against the untuned model on the same condition; % accuracy as a percentage of the untuned model’s clean-code accuracy in the same direction, i.e. how much of the original accuracy the system brings back (100 = the untuned model on clean code). *fmt*: format-failure rate above 0.25, the cell measures the prompt contract rather than the task and enters no mean. Pooled rows average the stacks above them. ‘-’: cell not run.

| Condition                                              | untuned |     | ICL        |          |     | clean LoRA |          |     | breadth  |          |     | anchored  |          |     | family   |          |     |
|--------------------------------------------------------|---------|-----|------------|----------|-----|------------|----------|-----|----------|----------|-----|-----------|----------|-----|----------|----------|-----|
|                                                        | base    |     | base_1shot |          |     | tuned_L0   |          |     | mono_all |          |     | cons_lam3 |          |     | tuned_X1 |          |     |
|                                                        | acc     | %   | acc        | $\Delta$ | %   | acc        | $\Delta$ | %   | acc      | $\Delta$ | %   | acc       | $\Delta$ | %   | acc      | $\Delta$ | %   |
| <i>Forward: output prediction</i>                      |         |     |            |          |     |            |          |     |          |          |     |           |          |     |          |          |     |
| L0                                                     | 0.285   | 100 | 0.293      | +0.8     | 103 | 0.430      | +14.5    | 151 | 0.416    | +13.1    | 146 | 0.396     | +11.1    | 139 | 0.413    | +12.8    | 145 |
| L1b                                                    | 0.267   | 94  | 0.267      | +0.0     | 94  | 0.338      | +7.1     | 118 | 0.391    | +12.5    | 137 | 0.381     | +11.4    | 134 | 0.321    | +5.4     | 113 |
| L1r                                                    | 0.268   | 94  | 0.260      | -0.8     | 91  | 0.375      | +10.8    | 132 | 0.383    | +11.5    | 134 | 0.378     | +11.1    | 133 | 0.376    | +10.8    | 132 |
| L2                                                     | 0.251   | 88  | 0.248      | -0.4     | 87  | 0.372      | +12.0    | 130 | 0.387    | +13.5    | 136 | 0.377     | +12.6    | 132 | 0.366    | +11.4    | 128 |
| S1                                                     | 0.254   | 89  | 0.253      | -0.2     | 89  | 0.346      | +9.1     | 121 | 0.379    | +12.5    | 133 | 0.358     | +10.4    | 126 | 0.350    | +9.5     | 123 |
| S2                                                     | 0.272   | 96  | 0.276      | +0.4     | 97  | 0.386      | +11.4    | 136 | 0.415    | +14.2    | 145 | 0.394     | +12.1    | 138 | 0.398    | +12.5    | 140 |
| X1                                                     | 0.147   | 51  | -          | -        | -   | 0.216      | +6.9     | 76  | 0.207    | +6.0     | 73  | 0.188     | +4.1     | 66  | 0.282    | +13.5    | 99  |
| C_L1b_S1                                               | 0.219   | 77  | -          | -        | -   | 0.263      | +4.4     | 92  | 0.342    | +12.4    | 120 | 0.337     | +11.8    | 118 | 0.261    | +4.2     | 92  |
| C_L1r_S1                                               | 0.204   | 72  | -          | -        | -   | 0.279      | +7.5     | 98  | 0.344    | +14.0    | 121 | 0.338     | +13.3    | 118 | 0.291    | +8.7     | 102 |
| C_S1_L1r                                               | 0.224   | 79  | -          | -        | -   | 0.279      | +5.4     | 98  | 0.326    | +10.2    | 115 | 0.319     | +9.5     | 112 | 0.291    | +6.6     | 102 |
| C_L2_S4                                                | 0.242   | 85  | -          | -        | -   | 0.334      | +9.2     | 117 | 0.376    | +13.4    | 132 | 0.376     | +13.4    | 132 | 0.341    | +9.9     | 120 |
| C_L1r_S3                                               | 0.259   | 91  | -          | -        | -   | 0.350      | +9.1     | 123 | 0.382    | +12.3    | 134 | 0.376     | +11.7    | 132 | 0.359    | +10.1    | 126 |
| C_S4_S3                                                | 0.270   | 95  | -          | -        | -   | 0.380      | +11.0    | 133 | 0.412    | +14.2    | 144 | 0.389     | +11.9    | 137 | 0.383    | +11.3    | 134 |
| seen stacks, pooled (6)                                | 0.236   | 83  | -          | -        | -   | 0.314      | +7.8     | 110 | 0.364    | +12.7    | 128 | 0.356     | +12.0    | 125 | 0.321    | +8.5     | 113 |
| C3_L1r_S3_S4                                           | -       | -   | -          | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| C3_S1_S3_S4                                            | -       | -   | -          | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| C3_L1r_S1_S4                                           | -       | -   | -          | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| C4_L1r_S1_S3_S4                                        | -       | -   | -          | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| depth-3/4 stacks, pooled (4)                           | -       | -   | -          | -        | -   | -          | -        | -   | -        | -        | -   | -         | -        | -   | -        | -        | -   |
| C_L1r_X1                                               | 0.136   | 48  | -          | -        | -   | 0.157      | +2.1     | 55  | 0.172    | +3.6     | 60  | 0.163     | +2.7     | 57  | 0.245    | +11.0    | 86  |
| C_X1_S1                                                | 0.153   | 54  | -          | -        | -   | 0.222      | +7.0     | 78  | 0.203    | +5.0     | 71  | 0.186     | +3.3     | 65  | 0.272    | +12.0    | 96  |
| C_S2_X1                                                | 0.209   | 73  | -          | -        | -   | 0.263      | +5.5     | 92  | 0.252    | +4.4     | 89  | 0.240     | +3.1     | 84  | 0.323    | +11.4    | 113 |
| unseen-containing stacks, pooled (3)                   | 0.166   | 58  | -          | -        | -   | 0.214      | +4.8     | 75  | 0.209    | +4.3     | 73  | 0.196     | +3.1     | 69  | 0.280    | +11.4    | 98  |
| C_L1r_X1m                                              | 0.158   | 55  | -          | -        | -   | 0.188      | +3.0     | 66  | 0.204    | +4.7     | 72  | 0.183     | +2.6     | 64  | 0.279    | +12.1    | 98  |
| C_S1_X1s                                               | 0.206   | 72  | -          | -        | -   | 0.298      | +9.2     | 104 | 0.267    | +6.1     | 94  | 0.269     | +6.3     | 94  | 0.318    | +11.2    | 111 |
| C3_L1r_S1_X1                                           | 0.167   | 59  | -          | -        | -   | 0.186      | +1.8     | 65  | 0.195    | +2.8     | 68  | 0.198     | +3.1     | 70  | 0.238    | +7.0     | 83  |
| <i>Backward: input prediction, graded by execution</i> |         |     |            |          |     |            |          |     |          |          |     |           |          |     |          |          |     |
| L0                                                     | 0.410   | 100 | -          | -        | -   | fmt        | fmt      | fmt | fmt      | fmt      | fmt | fmt       | fmt      | fmt | 0.280    | -13.0    | 68  |
| L1b                                                    | 0.362   | 88  | -          | -        | -   | fmt        | fmt      | fmt | fmt      | fmt      | fmt | fmt       | fmt      | fmt | fmt      | fmt      | fmt |
| L1r                                                    | 0.405   | 99  | -          | -        | -   | fmt        | fmt      | fmt | fmt      | fmt      | fmt | fmt       | fmt      | fmt | 0.312    | -9.3     | 76  |
| L2                                                     | 0.366   | 89  | -          | -        | -   | fmt        | fmt      | fmt | fmt      | fmt      | fmt | fmt       | fmt      | fmt | fmt      | fmt      | fmt |
| S1                                                     | 0.356   | 87  | -          | -        | -   | fmt        | fmt      | fmt | fmt      | fmt      | fmt | fmt       | fmt      | fmt | 0.264    | -9.1     | 65  |
| S2                                                     | 0.429   | 105 | -          | -        | -   | fmt        | fmt      | fmt | fmt      | fmt      | fmt | fmt       | fmt      | fmt | fmt      | fmt      | fmt |
| X1                                                     | 0.281   | 68  | -          | -        | -   | fmt        | fmt      | fmt | fmt      | fmt      | fmt | fmt       | fmt      | fmt | fmt      | fmt      | fmt |

Table 18. **Routing, CodeLlama-7B.** MoLE mixtures of the eight per-transform experts on every condition and stack that has them: *router* = trained gate, *hard router* = its argmax, *uniform* = fixed uniform gate, *random* = gate frozen at random initialisation. Per system: raw accuracy,  $\Delta$  points against the untuned model on the same items, and % of the untuned model’s clean-code accuracy. Every cell ran through the mixture engine.

| Condition       | untuned |     | router      |          |     | hard router     |          |     | uniform      |          |     | random      |          |     |
|-----------------|---------|-----|-------------|----------|-----|-----------------|----------|-----|--------------|----------|-----|-------------|----------|-----|
|                 | base    |     | mole_router |          |     | mole_hardrouter |          |     | mole_uniform |          |     | mole_random |          |     |
|                 | acc     | %   | acc         | $\Delta$ | %   | acc             | $\Delta$ | %   | acc          | $\Delta$ | %   | acc         | $\Delta$ | %   |
| L0              | 0.258   | 100 | 0.429       | +17.1    | 167 | 0.428           | +17.0    | 167 | 0.428        | +16.9    | 166 | 0.433       | +17.5    | 169 |
| L1b             | 0.196   | 76  | 0.381       | +18.5    | 148 | 0.384           | +18.8    | 149 | 0.371        | +17.5    | 144 | 0.375       | +17.9    | 146 |
| L1r             | 0.205   | 80  | 0.383       | +17.7    | 149 | 0.381           | +17.6    | 148 | 0.390        | +18.4    | 152 | 0.391       | +18.6    | 152 |
| L2              | 0.198   | 77  | 0.386       | +18.8    | 150 | 0.384           | +18.6    | 149 | 0.392        | +19.3    | 152 | 0.393       | +19.5    | 153 |
| S1              | 0.167   | 65  | 0.395       | +22.8    | 154 | 0.393           | +22.6    | 153 | 0.395        | +22.8    | 154 | 0.394       | +22.7    | 153 |
| S2              | 0.191   | 74  | 0.421       | +23.0    | 164 | 0.421           | +22.9    | 164 | 0.415        | +22.3    | 161 | 0.411       | +22.0    | 160 |
| C_L1b_S1        | 0.141   | 55  | 0.325       | +18.4    | 126 | 0.324           | +18.3    | 126 | 0.310        | +16.8    | 121 | 0.312       | +17.1    | 121 |
| C_L1r_S1        | 0.131   | 51  | 0.346       | +21.5    | 135 | 0.339           | +20.8    | 132 | 0.318        | +18.7    | 124 | 0.320       | +18.9    | 125 |
| C_S1_L1r        | 0.135   | 53  | 0.318       | +18.4    | 124 | 0.324           | +18.9    | 126 | 0.302        | +16.8    | 118 | 0.303       | +16.8    | 118 |
| C_L2_S4         | 0.176   | 68  | 0.383       | +20.8    | 149 | 0.379           | +20.3    | 148 | 0.380        | +20.4    | 148 | 0.379       | +20.3    | 147 |
| C_L1r_S3        | 0.172   | 67  | 0.391       | +21.9    | 152 | 0.389           | +21.7    | 151 | 0.383        | +21.1    | 149 | 0.380       | +20.8    | 148 |
| C_S4_S3         | 0.185   | 72  | 0.415       | +22.9    | 161 | 0.413           | +22.7    | 161 | 0.409        | +22.4    | 159 | 0.409       | +22.4    | 159 |
| C3_L1r_S3_S4    | 0.150   | 58  | 0.375       | +22.5    | 146 | 0.376           | +22.6    | 146 | 0.365        | +21.5    | 142 | 0.363       | +21.3    | 141 |
| C3_S1_S3_S4     | 0.128   | 50  | 0.377       | +24.8    | 147 | 0.378           | +24.9    | 147 | 0.367        | +23.8    | 143 | 0.367       | +23.9    | 143 |
| C3_L1r_S1_S4    | 0.126   | 49  | 0.325       | +19.9    | 126 | 0.326           | +20.0    | 127 | 0.306        | +18.0    | 119 | 0.305       | +17.9    | 119 |
| C4_L1r_S1_S3_S4 | 0.089   | 35  | 0.334       | +24.5    | 130 | 0.336           | +24.7    | 131 | 0.296        | +20.7    | 115 | 0.298       | +20.9    | 116 |
| C_L1r_X1        | 0.102   | 40  | 0.189       | +8.7     | 74  | 0.186           | +8.4     | 72  | 0.194        | +9.2     | 76  | 0.190       | +8.8     | 74  |
| C_X1_S1         | 0.104   | 40  | 0.291       | +18.7    | 113 | 0.286           | +18.2    | 111 | 0.293        | +18.9    | 114 | 0.295       | +19.1    | 115 |
| C_S2_X1         | 0.145   | 56  | 0.326       | +18.1    | 127 | 0.328           | +18.3    | 128 | 0.329        | +18.4    | 128 | 0.329       | +18.4    | 128 |
| C_L1r_X1m       | 0.114   | 44  | 0.227       | +11.3    | 88  | 0.224           | +11.0    | 87  | 0.221        | +10.7    | 86  | 0.221       | +10.6    | 86  |
| C_S1_X1s        | 0.148   | 58  | 0.350       | +20.2    | 136 | 0.348           | +20.0    | 135 | 0.344        | +19.6    | 134 | 0.343       | +19.5    | 134 |
| C3_L1r_S1_X1    | 0.102   | 40  | 0.242       | +14.0    | 94  | 0.245           | +14.3    | 95  | 0.234        | +13.2    | 91  | 0.239       | +13.8    | 93  |

Table 19. **Merging, CodeLlama-7B.** Weight-space merges of the per-transform specialists (TIES, DARE-TIES, DARE-linear) and two merges anchored on the clean-code adapter, on every condition and stack that has them. Same three numbers as the master grid.

| Condition       | untuned |     | TIES       |          |     | DARE-TIES       |          |     | DARE-linear       |          |   | L0-anch. TIES |          |     | L0-anch. DARE-TIES |          |     |
|-----------------|---------|-----|------------|----------|-----|-----------------|----------|-----|-------------------|----------|---|---------------|----------|-----|--------------------|----------|-----|
|                 | base    |     | merge_ties |          |     | merge_dare_ties |          |     | merge_dare_linear |          |   | l0merge_ties  |          |     | l0merge_dare_ties  |          |     |
|                 | acc     | %   | acc        | $\Delta$ | %   | acc             | $\Delta$ | %   | acc               | $\Delta$ | % | acc           | $\Delta$ | %   | acc                | $\Delta$ | %   |
| L0              | 0.257   | 100 | 0.367      | +11.0    | 143 | 0.422           | +16.5    | 164 | fmt               | fmt      |   | 0.394         | +13.7    | 153 | 0.413              | +15.6    | 161 |
| L1b             | 0.197   | 77  | 0.301      | +10.4    | 117 | 0.370           | +17.3    | 144 | fmt               | fmt      |   | 0.331         | +13.4    | 129 | 0.349              | +15.1    | 136 |
| L1r             | 0.207   | 81  | 0.324      | +11.7    | 126 | 0.381           | +17.4    | 148 | fmt               | fmt      |   | 0.338         | +13.1    | 132 | 0.352              | +14.5    | 137 |
| L2              | 0.202   | 79  | 0.318      | +11.6    | 124 | 0.375           | +17.3    | 146 | fmt               | fmt      |   | 0.337         | +13.5    | 131 | 0.355              | +15.3    | 138 |
| S1              | 0.168   | 66  | 0.321      | +15.2    | 125 | 0.374           | +20.6    | 146 | fmt               | fmt      |   | 0.342         | +17.3    | 133 | 0.365              | +19.6    | 142 |
| S2              | 0.193   | 75  | 0.341      | +14.8    | 133 | 0.396           | +20.3    | 154 | fmt               | fmt      |   | 0.365         | +17.2    | 142 | 0.379              | +18.5    | 147 |
| C_L1b_S1        | 0.141   | 55  | 0.245      | +10.4    | 95  | 0.298           | +15.7    | 116 | fmt               | fmt      |   | 0.269         | +12.8    | 105 | 0.280              | +13.9    | 109 |
| C_L1r_S1        | 0.131   | 51  | 0.252      | +12.1    | 98  | 0.301           | +17.0    | 117 | fmt               | fmt      |   | 0.277         | +14.6    | 108 | 0.283              | +15.2    | 110 |
| C_S1_L1r        | 0.135   | 53  | 0.240      | +10.5    | 94  | 0.302           | +16.8    | 118 | fmt               | fmt      |   | 0.273         | +13.8    | 106 | 0.282              | +14.7    | 110 |
| C_L2_S4         | 0.176   | 68  | 0.298      | +12.2    | 116 | 0.367           | +19.1    | 143 | fmt               | fmt      |   | 0.325         | +14.9    | 126 | 0.349              | +17.3    | 136 |
| C_L1r_S3        | 0.172   | 67  | 0.306      | +13.4    | 119 | 0.367           | +19.5    | 143 | fmt               | fmt      |   | 0.326         | +15.4    | 127 | 0.338              | +16.6    | 131 |
| C_S4_S3         | 0.186   | 72  | 0.336      | +15.0    | 131 | 0.395           | +20.9    | 154 | fmt               | fmt      |   | 0.367         | +18.1    | 143 | 0.385              | +19.9    | 150 |
| C3_L1r_S3_S4    | 0.150   | 58  | 0.296      | +14.6    | 115 | 0.353           | +20.3    | 137 | fmt               | fmt      |   | 0.313         | +16.3    | 122 | 0.328              | +17.8    | 128 |
| C3_S1_S3_S4     | 0.128   | 50  | 0.274      | +14.5    | 107 | 0.340           | +21.1    | 132 | fmt               | fmt      |   | 0.295         | +16.6    | 115 | 0.315              | +18.7    | 123 |
| C3_L1r_S1_S4    | 0.126   | 49  | 0.239      | +11.3    | 93  | 0.298           | +17.2    | 116 | fmt               | fmt      |   | 0.272         | +14.6    | 106 | 0.283              | +15.7    | 110 |
| C4_L1r_S1_S3_S4 | 0.089   | 35  | 0.214      | +12.5    | 83  | 0.291           | +20.1    | 113 | fmt               | fmt      |   | 0.243         | +15.4    | 95  | 0.256              | +16.7    | 100 |
| C_L1r_X1        | 0.102   | 40  | -          | -        | -   | 0.192           | +9.0     | 75  | -                 | -        |   | -             | -        | -   | -                  | -        | -   |
| C_X1_S1         | 0.104   | 40  | -          | -        | -   | 0.287           | +18.3    | 112 | -                 | -        |   | -             | -        | -   | -                  | -        | -   |
| C_S2_X1         | 0.145   | 57  | -          | -        | -   | 0.316           | +17.1    | 123 | -                 | -        |   | -             | -        | -   | -                  | -        | -   |
| C_L1r_X1m       | 0.112   | 44  | -          | -        | -   | 0.212           | +10.0    | 83  | -                 | -        |   | -             | -        | -   | -                  | -        | -   |
| C_S1_X1s        | 0.151   | 59  | -          | -        | -   | 0.336           | +18.5    | 131 | -                 | -        |   | -             | -        | -   | -                  | -        | -   |
| C3_L1r_S1_X1    | 0.104   | 40  | -          | -        | -   | 0.234           | +13.0    | 91  | -                 | -        |   | -             | -        | -   | -                  | -        | -   |

Table 20. **Merging, Granite-3.1-8B.** Weight-space merges of the per-transform specialists (TIES, DARE-TIES, DARE-linear) and two merges anchored on the clean-code adapter, on every condition and stack that has them. Same three numbers as the master grid.

| Condition    | untuned base |     | TIES       |      |          | DARE-TIES       |       |          | DARE-linear       |     |          | L0-anch. TIES |     |          | L0-anch. DARE-TIES |     |          |
|--------------|--------------|-----|------------|------|----------|-----------------|-------|----------|-------------------|-----|----------|---------------|-----|----------|--------------------|-----|----------|
|              | acc          | %   | merge_ties | acc  | $\Delta$ | merge_dare_ties | acc   | $\Delta$ | merge_dare_linear | acc | $\Delta$ | l0merge_ties  | acc | $\Delta$ | l0merge_dare_ties  | acc | $\Delta$ |
| L0           | 0.285        | 100 | 0.383      | +9.8 | 134      | 0.433           | +14.8 | 152      | fmt               | fmt | fmt      | -             | -   | -        | -                  | -   | -        |
| L1b          | 0.266        | 93  | 0.315      | +4.9 | 110      | 0.350           | +8.4  | 123      | fmt               | fmt | fmt      | -             | -   | -        | -                  | -   | -        |
| L1r          | 0.268        | 94  | 0.353      | +8.5 | 124      | 0.384           | +11.6 | 135      | fmt               | fmt | fmt      | -             | -   | -        | -                  | -   | -        |
| L2           | 0.251        | 88  | 0.342      | +9.0 | 120      | 0.378           | +12.6 | 133      | fmt               | fmt | fmt      | -             | -   | -        | -                  | -   | -        |
| S1           | 0.254        | 89  | 0.323      | +6.9 | 113      | 0.371           | +11.7 | 130      | fmt               | fmt | fmt      | -             | -   | -        | -                  | -   | -        |
| S2           | 0.272        | 96  | 0.358      | +8.5 | 125      | 0.402           | +13.0 | 141      | fmt               | fmt | fmt      | -             | -   | -        | -                  | -   | -        |
| X1           | 0.147        | 51  | 0.199      | +5.3 | 70       | 0.231           | +8.4  | 81       | fmt               | fmt | fmt      | -             | -   | -        | -                  | -   | -        |
| C_L1b_S1     | 0.219        | 77  | 0.247      | +2.9 | 87       | 0.279           | +6.0  | 98       | fmt               | fmt | fmt      | -             | -   | -        | -                  | -   | -        |
| C_L1r_S1     | 0.204        | 72  | 0.266      | +6.1 | 93       | 0.301           | +9.7  | 106      | fmt               | fmt | fmt      | -             | -   | -        | -                  | -   | -        |
| C_S1_L1r     | 0.224        | 79  | 0.262      | +3.8 | 92       | 0.293           | +6.9  | 103      | fmt               | fmt | fmt      | -             | -   | -        | -                  | -   | -        |
| C_L2_S4      | 0.242        | 85  | 0.319      | +7.7 | 112      | 0.351           | +10.9 | 123      | fmt               | fmt | fmt      | -             | -   | -        | -                  | -   | -        |
| C_L1r_S3     | 0.259        | 91  | 0.324      | +6.5 | 114      | 0.365           | +10.6 | 128      | fmt               | fmt | fmt      | -             | -   | -        | -                  | -   | -        |
| C_S4_S3      | 0.270        | 95  | 0.353      | +8.3 | 124      | 0.394           | +12.4 | 138      | fmt               | fmt | fmt      | -             | -   | -        | -                  | -   | -        |
| C_L1r_X1     | 0.136        | 48  | 0.153      | +1.7 | 54       | 0.173           | +3.7  | 61       | fmt               | fmt | fmt      | -             | -   | -        | -                  | -   | -        |
| C_X1_S1      | 0.153        | 54  | 0.200      | +4.7 | 70       | 0.244           | +9.1  | 86       | fmt               | fmt | fmt      | -             | -   | -        | -                  | -   | -        |
| C_S2_X1      | 0.209        | 73  | 0.246      | +3.7 | 86       | 0.272           | +6.3  | 95       | fmt               | fmt | fmt      | -             | -   | -        | -                  | -   | -        |
| C3_L1r_S1_X1 | 0.167        | 59  | 0.204      | +3.7 | 72       | 0.217           | +5.0  | 76       | fmt               | fmt | fmt      | -             | -   | -        | -                  | -   | -        |

Table 21. **Merging, Llama-3.1-8B.** Weight-space merges of the per-transform specialists (TIES, DARE-TIES, DARE-linear) and two merges anchored on the clean-code adapter, on every condition and stack that has them. Same three numbers as the master grid.

| Condition    | untuned base |     | TIES       |       |          | DARE-TIES       |       |          | DARE-linear       |      |          | L0-anch. TIES |     |          | L0-anch. DARE-TIES |     |          |
|--------------|--------------|-----|------------|-------|----------|-----------------|-------|----------|-------------------|------|----------|---------------|-----|----------|--------------------|-----|----------|
|              | acc          | %   | merge_ties | acc   | $\Delta$ | merge_dare_ties | acc   | $\Delta$ | merge_dare_linear | acc  | $\Delta$ | l0merge_ties  | acc | $\Delta$ | l0merge_dare_ties  | acc | $\Delta$ |
| L0           | 0.259        | 100 | 0.398      | +14.0 | 154      | 0.462           | +20.3 | 179      | 0.292             | +3.4 | 113      | -             | -   | -        | -                  | -   | -        |
| L1b          | 0.222        | 86  | 0.336      | +11.4 | 130      | 0.386           | +16.4 | 150      | 0.253             | +3.1 | 98       | -             | -   | -        | -                  | -   | -        |
| L1r          | 0.242        | 94  | 0.348      | +10.6 | 135      | 0.398           | +15.6 | 154      | 0.234             | -0.8 | 91       | -             | -   | -        | -                  | -   | -        |
| L2           | 0.215        | 83  | 0.353      | +13.8 | 137      | 0.396           | +18.1 | 153      | 0.246             | +3.1 | 95       | -             | -   | -        | -                  | -   | -        |
| S1           | 0.213        | 83  | 0.335      | +12.2 | 130      | 0.412           | +19.9 | 160      | 0.256             | +4.3 | 99       | -             | -   | -        | -                  | -   | -        |
| S2           | 0.208        | 80  | 0.361      | +15.3 | 140      | 0.437           | +23.0 | 169      | 0.256             | +4.9 | 99       | -             | -   | -        | -                  | -   | -        |
| X1           | 0.132        | 51  | 0.217      | +8.5  | 84       | 0.266           | +13.4 | 103      | fmt               | fmt  | fmt      | -             | -   | -        | -                  | -   | -        |
| C_L1b_S1     | 0.152        | 59  | 0.252      | +10.1 | 98       | 0.307           | +15.6 | 119      | 0.211             | +6.0 | 82       | -             | -   | -        | -                  | -   | -        |
| C_L1r_S1     | 0.162        | 63  | 0.273      | +11.1 | 106      | 0.328           | +16.6 | 127      | 0.206             | +4.4 | 80       | -             | -   | -        | -                  | -   | -        |
| C_S1_L1r     | 0.159        | 62  | 0.247      | +8.9  | 96       | 0.306           | +14.8 | 119      | 0.200             | +4.2 | 78       | -             | -   | -        | -                  | -   | -        |
| C_L2_S4      | 0.206        | 80  | 0.311      | +10.5 | 120      | 0.383           | +17.8 | 149      | 0.226             | +2.0 | 88       | -             | -   | -        | -                  | -   | -        |
| C_L1r_S3     | 0.198        | 77  | 0.334      | +13.5 | 129      | 0.387           | +18.9 | 150      | 0.222             | +2.3 | 86       | -             | -   | -        | -                  | -   | -        |
| C_S4_S3      | 0.214        | 83  | 0.357      | +14.3 | 138      | 0.437           | +22.3 | 169      | 0.255             | +4.1 | 99       | -             | -   | -        | -                  | -   | -        |
| C_L1r_X1     | 0.100        | 39  | 0.147      | +4.6  | 57       | 0.184           | +8.3  | 71       | fmt               | fmt  | fmt      | -             | -   | -        | -                  | -   | -        |
| C_X1_S1      | 0.130        | 50  | 0.233      | +10.3 | 90       | 0.296           | +16.6 | 115      | fmt               | fmt  | fmt      | -             | -   | -        | -                  | -   | -        |
| C_S2_X1      | 0.169        | 66  | 0.263      | +9.4  | 102      | 0.317           | +14.8 | 123      | fmt               | fmt  | fmt      | -             | -   | -        | -                  | -   | -        |
| C3_L1r_S1_X1 | 0.105        | 41  | 0.170      | +6.6  | 66       | 0.215           | +11.0 | 83       | fmt               | fmt  | fmt      | -             | -   | -        | -                  | -   | -        |

Table 22. **Merging, StarCoder2-15B**. Weight-space merges of the per-transform specialists (TIES, DARE-TIES, DARE-linear) and two merges anchored on the clean-code adapter, on every condition and stack that has them. Same three numbers as the master grid.

| Condition    | untuned base |     | TIES  |     |     | DARE-TIES |     |     | DARE-linear |     |     | l0merge_ties |   |   | l0merge_dare_ties |   |   |
|--------------|--------------|-----|-------|-----|-----|-----------|-----|-----|-------------|-----|-----|--------------|---|---|-------------------|---|---|
|              | acc          | %   | acc   | Δ   | %   | acc       | Δ   | %   | acc         | Δ   | %   | acc          | Δ | % | acc               | Δ | % |
| L0           | fnt          | fnt | 0.484 | fnt | fnt | 0.539     | fnt | fnt | fnt         | fnt | fnt | -            | - | - | -                 | - | - |
| L1b          | fnt          | fnt | 0.366 | fnt | fnt | 0.426     | fnt | fnt | fnt         | fnt | fnt | -            | - | - | -                 | - | - |
| L1r          | fnt          | fnt | 0.414 | fnt | fnt | 0.476     | fnt | fnt | fnt         | fnt | fnt | -            | - | - | -                 | - | - |
| L2           | fnt          | fnt | 0.401 | fnt | fnt | 0.466     | fnt | fnt | fnt         | fnt | fnt | -            | - | - | -                 | - | - |
| S1           | fnt          | fnt | 0.451 | fnt | fnt | 0.487     | fnt | fnt | fnt         | fnt | fnt | -            | - | - | -                 | - | - |
| S2           | fnt          | fnt | 0.440 | fnt | fnt | 0.518     | fnt | fnt | fnt         | fnt | fnt | -            | - | - | -                 | - | - |
| X1           | fnt          | fnt | 0.283 | fnt | fnt | 0.343     | fnt | fnt | fnt         | fnt | fnt | -            | - | - | -                 | - | - |
| C_L1b_S1     | fnt          | fnt | 0.321 | fnt | fnt | 0.354     | fnt | fnt | 0.063       | fnt | fnt | -            | - | - | -                 | - | - |
| C_L1r_S1     | fnt          | fnt | 0.344 | fnt | fnt | 0.389     | fnt | fnt | 0.047       | fnt | fnt | -            | - | - | -                 | - | - |
| C_S1_L1r     | fnt          | fnt | 0.346 | fnt | fnt | 0.387     | fnt | fnt | 0.041       | fnt | fnt | -            | - | - | -                 | - | - |
| C_L2_S4      | fnt          | fnt | 0.381 | fnt | fnt | 0.438     | fnt | fnt | fnt         | fnt | fnt | -            | - | - | -                 | - | - |
| C_L1r_S3     | fnt          | fnt | 0.390 | fnt | fnt | 0.459     | fnt | fnt | fnt         | fnt | fnt | -            | - | - | -                 | - | - |
| C_S4_S3      | fnt          | fnt | 0.442 | fnt | fnt | 0.506     | fnt | fnt | fnt         | fnt | fnt | -            | - | - | -                 | - | - |
| C_L1r_X1     | fnt          | fnt | 0.215 | fnt | fnt | 0.259     | fnt | fnt | 0.058       | fnt | fnt | -            | - | - | -                 | - | - |
| C_X1_S1      | fnt          | fnt | 0.295 | fnt | fnt | 0.371     | fnt | fnt | 0.067       | fnt | fnt | -            | - | - | -                 | - | - |
| C_S2_X1      | fnt          | fnt | 0.341 | fnt | fnt | 0.403     | fnt | fnt | fnt         | fnt | fnt | -            | - | - | -                 | - | - |
| C3_L1r_S1_X1 | fnt          | fnt | 0.241 | fnt | fnt | 0.286     | fnt | fnt | 0.058       | fnt | fnt | -            | - | - | -                 | - | - |

Table 23. **RQ1: routing and merging on stacks of seen transforms**. Paired differences in points on the ten stacks (six depth-2, four depth-3/4), CodeLlama-7B, 394 programs common to every cell; the last column repeats the contrasts on the four stacks that contain the unseen family (319 programs; merges other than DARE-TIES have no cells there). Routing is worth about a point over a random gate and nothing over breadth; the mixture itself carries the gain (mole\_uniform – base); a hard argmax router is indistinguishable from the soft one. The best merge (DARE-TIES) matches the clean-code control and the other four fall below it. Bold intervals exclude zero. Every arm ran through the mixture engine (obtune.mole.eval\_mole); an earlier run through vLLM had written the untuned model under the routing arms’ names and was withdrawn.

| Contrast                      | all ten seen            | depth 2                 | depth 3–4               | unseen inside           |
|-------------------------------|-------------------------|-------------------------|-------------------------|-------------------------|
| <i>Routing</i>                |                         |                         |                         |                         |
| mole_router – mole_random     | +1.34 [+0.48, +2.17]    | +1.11 [+0.28, +1.93]    | +1.67 [+0.47, +2.86]    | –0.03 [–0.99, +0.84]    |
| mole_router – mole_uniform    | +1.40 [+0.52, +2.25]    | +1.17 [+0.31, +2.02]    | +1.74 [+0.53, +2.90]    | +0.10 [–0.84, +0.99]    |
| mole_hardrouter – mole_router | +0.03 [–0.13, +0.19]    | –0.03 [–0.24, +0.18]    | +0.13 [–0.11, +0.38]    | –0.13 [–0.44, +0.18]    |
| mole_router – mono_all        | –0.28 [–1.57, +1.09]    | –0.06 [–1.38, +1.30]    | –0.61 [–2.10, +0.91]    | +2.88 [+1.33, +4.41]    |
| mole_uniform – base           | +19.79 [+17.40, +22.23] | +19.16 [+16.77, +21.58] | +20.72 [+18.16, +23.31] | +14.98 [+12.81, +17.29] |
| <i>Merging</i>                |                         |                         |                         |                         |
| merge_dare_ties – tuned_L0    | +0.52 [–0.15, +1.20]    | +0.61 [–0.16, +1.41]    | +0.38 [–0.47, +1.25]    | +0.86 [–0.05, +1.78]    |
| merge_ties – tuned_L0         | –5.50 [–6.86, –4.13]    | –5.19 [–6.59, –3.77]    | –5.95 [–7.47, –4.44]    | –                       |
| merge_dare_linear – tuned_L0  | –20.56 [–22.88, –18.25] | –20.60 [–23.00, –18.22] | –20.49 [–22.86, –18.15] | –                       |
| l0merge_ties – tuned_L0       | –3.14 [–4.21, –2.12]    | –2.81 [–3.98, –1.67]    | –3.64 [–4.91, –2.41]    | –                       |
| l0merge_dare_ties – tuned_L0  | –1.74 [–2.57, –0.91]    | –1.40 [–2.30, –0.47]    | –2.24 [–3.26, –1.23]    | –                       |
| <i>Breadth, for reference</i> |                         |                         |                         |                         |
| mono_all – tuned_L0           | +3.67 [+2.02, +5.24]    | +3.34 [+1.61, +5.04]    | +4.15 [+2.35, +5.91]    | –1.49 [–3.08, +0.08]    |

Table 24. **Weight-space merging on the panel.** The best merge (DARE-TIES of the clean adapter and the five specialists) against the clean-code control, breadth and the anchored objective, pooled over the six seen stacks and the four stacks containing the unseen family; breadth and anchoring against the control in the last two columns. Points, paired on the programs common to every cell, program-clustered bootstrap. Bold intervals exclude zero. Pre-registered per model: merging does not beat the control on seen stacks (CONFIRMED iff  $ci_{hi} \leq +1$ ) and stays below breadth ( $ci_{hi} < 0$ ).

| Stacks                | merge-clean          | merge-mono           | merge-cons           | mono-clean           | cons-clean           |
|-----------------------|----------------------|----------------------|----------------------|----------------------|----------------------|
| <b>CodeLlama-7B</b>   |                      |                      |                      |                      |                      |
| seen (6)              | +0.83 [+0.01, +1.63] | -2.80 [-4.43, -1.19] | -4.01 [-5.29, -2.70] | +3.63 [+1.96, +5.27] | +4.84 [+3.49, +6.21] |
| unseen inside (4)     | +0.86 [-0.05, +1.78] | +2.35 [+0.63, +3.97] | -1.39 [-2.72, -0.05] | -1.49 [-3.08, +0.08] | +2.25 [+0.86, +3.61] |
| <b>Granite-3.1-8B</b> |                      |                      |                      |                      |                      |
| seen (6)              | +1.57 [+0.76, +2.46] | -3.23 [-4.92, -1.52] | -2.32 [-4.01, -0.61] | +4.80 [+3.05, +6.53] | +3.89 [+2.12, +5.67] |
| unseen inside (4)     | +2.09 [+1.10, +3.09] | +2.20 [+0.50, +3.84] | +2.98 [+1.28, +4.65] | -0.10 [-1.78, +1.62] | -0.89 [-2.59, +0.89] |
| <b>Llama-3.1-8B</b>   |                      |                      |                      |                      |                      |
| seen (6)              | +2.31 [+1.27, +3.38] | -0.85 [-2.41, +0.72] | -2.80 [-4.26, -1.35] | +3.16 [+1.52, +4.84] | +5.11 [+3.63, +6.58] |
| unseen inside (4)     | +2.56 [+1.49, +3.61] | +3.09 [+1.54, +4.60] | +0.81 [-0.47, +2.12] | -0.52 [-2.07, +1.02] | +1.75 [+0.39, +3.11] |
| <b>StarCoder2-15B</b> |                      |                      |                      |                      |                      |
| seen (6)              | -0.69 [-1.57, +0.13] | -3.27 [-4.67, -1.86] | -5.73 [-7.05, -4.47] | +2.57 [+1.28, +3.86] | +5.04 [+3.82, +6.26] |
| unseen inside (4)     | +1.15 [+0.08, +2.17] | +1.70 [+0.05, +3.32] | +0.39 [-0.92, +1.80] | -0.55 [-2.12, +1.02] | +0.76 [-0.47, +2.02] |

Table 25. **The direction ratio: how much of a forward gain survives the task being run backwards.** Forward is output prediction; backward is input prediction on the same programs, graded by execution. No adapter is trained on the backward task, so any backward gain is transferred program understanding rather than task fitting.  $DR < 0$  means the arm bought forward accuracy by *losing* backward competence. Every adaptation method that fits the forward task directly is negative; only anchoring and family exposure are not. Both directions use the same 412 programs and the same six conditions. Bold intervals exclude zero; the ratio is descriptive and is read with its components.

| Method                    | Arm        | Forward gain            | Backward gain        | DR    |
|---------------------------|------------|-------------------------|----------------------|-------|
| Format-only control       | formatonly | +1.47 [+0.90, +2.10]    | -0.44 [-0.81, -0.08] | -0.30 |
| Clean-code tuning         | tuned_L0   | +19.34 [+16.87, +21.81] | -1.60 [-3.41, +0.35] | -0.08 |
| Training breadth          | mono_all   | +19.46 [+16.68, +22.01] | -0.89 [-2.86, +1.04] | -0.05 |
| Paired consistency (ours) | cons_lam3  | +20.88 [+18.07, +23.49] | +0.62 [-1.56, +2.81] | +0.03 |
| Family exposure           | tuned_X1   | +17.35 [+14.98, +19.73] | +1.63 [+0.11, +3.20] | +0.09 |

Table 26. **RQ4: the divergence result across the panel**, on stacks containing the unseen family. cons-mono is anchoring’s advantage over breadth; fam is an arm trained on the unseen family’s trainable sibling, included because family exposure is the largest effect in the table and replicates on every model. Bold intervals exclude zero.

| Model          | Lineage | cons-mono            | cons-clean           | fam-clean            |
|----------------|---------|----------------------|----------------------|----------------------|
| CodeLlama-7B   | Meta    | +3.84 [+1.90, +5.70] | +2.13 [+0.66, +3.60] | +5.93 [+4.10, +7.78] |
| CodeLlama-13B  | Meta    | +3.22 [+1.74, +4.65] | +2.02 [+0.58, +3.53] | +5.97 [+4.11, +7.91] |
| CodeLlama-34B  | Meta    | +3.95 [+2.13, +5.81] | +3.10 [+1.63, +4.58] | +4.19 [+2.48, +5.94] |
| Llama-3.1-8B   | Meta    | +2.87 [+1.20, +4.62] | +1.86 [+0.27, +3.53] | +6.67 [+4.58, +8.71] |
| StarCoder2-15B | BigCode | +1.36 [-0.08, +2.71] | +0.35 [-1.13, +1.86] | +3.64 [+1.90, +5.35] |
| Gemma-3-12B    | Google  | +2.44 [+0.97, +3.99] | -0.08 [-1.67, +1.51] | +5.35 [+3.21, +7.37] |
| CodeGemma-7B   | Google  | -1.59 [-3.26, +0.12] | -2.40 [-4.11, -0.85] | +5.23 [+3.25, +7.21] |
| Granite-3.1-8B | IBM     | -1.47 [-2.99, +0.00] | -1.86 [-3.80, +0.12] | +7.29 [+5.19, +9.65] |

Table 27. **Every intervention against every kind of obfuscation, across the eight-model panel.** For each system and direction,  $\Delta$  is the mean change in points against the untuned model and % the mean accuracy as a percentage of the untuned model’s clean-code accuracy, over the cells that exist and pass the format gate (in parentheses: the number of cells; after the semicolon, the number gated as *fnt*). Systems: ICL = base\_1shot, clean LoRA = tuned\_L0, breadth = mono\_all, anchored = cons\_lam3, family = tuned\_X1. *seen* = L1b, L1r, L2, S1, S2; *seen stacks* = the six depth-2 and four depth-3/4 stacks of seen transforms; *unseen stacks* = the three depth-2 and one depth-3 stacks with the X1 family inside. Backward cells survive the gate almost only on CodeLlama-7B, whose grid (Table 10) is the one fully interpretable backward read; the per-model grids for the whole panel are in Appendix A.

| System     | measure          | clean            | seen               | unseen family    | seen stacks        | unseen stacks     |
|------------|------------------|------------------|--------------------|------------------|--------------------|-------------------|
| ICL        | fwd $\Delta$ pts | +3.8 (3; 5 fnt)  | +3.2 (16; 24 fnt)  | –                | –                  | –                 |
|            | fwd % of clean   | 115% (3; 5 fnt)  | 97% (16; 24 fnt)   | –                | –                  | –                 |
| clean LoRA | fwd $\Delta$ pts | +19.0 (5; 3 fnt) | +18.5 (30; 10 fnt) | +13.7 (7; 1 fnt) | +16.7 (40; 12 fnt) | +11.5 (28; 4 fnt) |
|            | fwd % of clean   | 175% (5; 3 fnt)  | 152% (25; 15 fnt)  | 102% (5; 3 fnt)  | 131% (34; 18 fnt)  | 95% (20; 12 fnt)  |
|            | bwd $\Delta$ pts | –3.8 (4; 4 fnt)  | –0.2 (17; 23 fnt)  | +0.2 (3; 5 fnt)  | –                  | –                 |
|            | bwd % of clean   | 93% (4; 4 fnt)   | 91% (17; 23 fnt)   | 65% (3; 5 fnt)   | –                  | –                 |
| breadth    | fwd $\Delta$ pts | +17.0 (5; 3 fnt) | +19.7 (30; 10 fnt) | +10.7 (7; 1 fnt) | +20.6 (40; 12 fnt) | +10.9 (28; 4 fnt) |
|            | fwd % of clean   | 167% (5; 3 fnt)  | 157% (25; 15 fnt)  | 91% (5; 3 fnt)   | 146% (34; 18 fnt)  | 93% (20; 12 fnt)  |
|            | bwd $\Delta$ pts | –2.4 (2; 6 fnt)  | –2.1 (12; 28 fnt)  | +0.2 (2; 6 fnt)  | –                  | –                 |
|            | bwd % of clean   | 95% (2; 6 fnt)   | 89% (12; 28 fnt)   | 76% (2; 6 fnt)   | –                  | –                 |
| anchored   | fwd $\Delta$ pts | +17.6 (5; 3 fnt) | +21.0 (30; 10 fnt) | +13.1 (7; 1 fnt) | +21.6 (40; 12 fnt) | +12.8 (28; 4 fnt) |
|            | fwd % of clean   | 169% (5; 3 fnt)  | 161% (25; 15 fnt)  | 99% (5; 3 fnt)   | 149% (34; 18 fnt)  | 98% (20; 12 fnt)  |
|            | bwd $\Delta$ pts | +1.3 (1; 7 fnt)  | +0.4 (5; 35 fnt)   | +2.5 (1; 7 fnt)  | –                  | –                 |
|            | bwd % of clean   | 104% (1; 7 fnt)  | 103% (5; 35 fnt)   | 99% (1; 7 fnt)   | –                  | –                 |
| family     | fwd $\Delta$ pts | +18.5 (5; 3 fnt) | +17.8 (30; 10 fnt) | +20.1 (7; 1 fnt) | +16.9 (30; 12 fnt) | +16.8 (28; 4 fnt) |
|            | fwd % of clean   | 173% (5; 3 fnt)  | 150% (25; 15 fnt)  | 128% (5; 3 fnt)  | 135% (24; 18 fnt)  | 116% (20; 12 fnt) |
|            | bwd $\Delta$ pts | –5.5 (5; 3 fnt)  | –1.2 (20; 20 fnt)  | +5.2 (3; 5 fnt)  | –                  | –                 |
|            | bwd % of clean   | 88% (5; 3 fnt)   | 89% (20; 20 fnt)   | 86% (3; 5 fnt)   | –                  | –                 |